Operating Systems

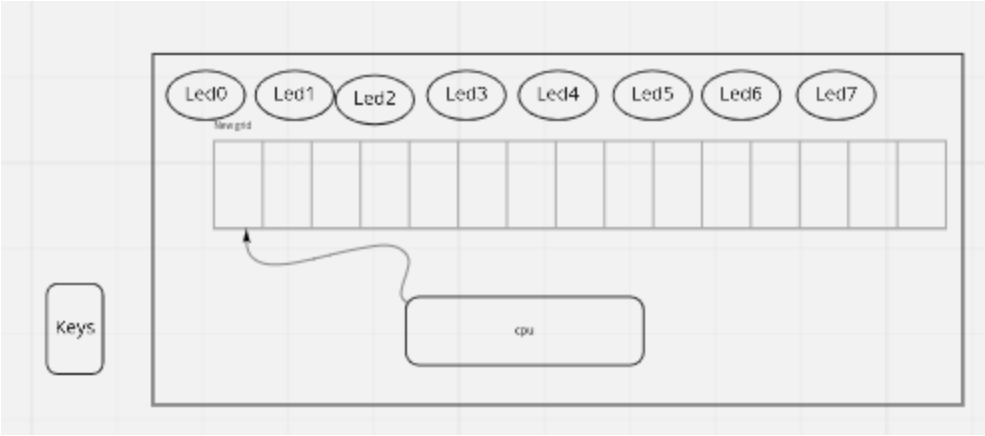
Ahmad Yoosofan

University of Kashan

https://yoosofan.github.io/en/

Yoosofan Imaginary Computer (YIC)

Based on Morris Mano's famous book


Led0
Led1
Led2
Led3
Led4
Led5
Led6
Led7
Keys
cpu

YIC 10, Instruction Set(I)

```
AND: Logical AND memory with AC
ADD: Arithmetic ADD memory with AC
LDA: Load from memory to AC
STA: Store AC to memory
BUN: Branch unconditional
ISZ: Increment and skip if zero
CLA: Clear AC
CLE: Clear E
CMA: Complement AC
CME: Complement E
CIR: Circulate right (AC and E)
CIL: Circulate left (AC and E)
```

```
INC:    Increment AC
SPA:    Skip if positive AC
SNA:    Skip if negative AC
SZA:    Skip if zero AC
SZE:    Skip if zero E
HLT:    Halt
OUT:    Output a character from AC
SKO:    Skip if output flag
NOP:    No operation
```

YIC 10, Instruction Set Bianary(II)

AND:	00001
ADD:	00010
LDA:	00011
STA:	00100
BUN:	00101
ISZ:	00110
CLA:	00111
CLE:	01000
CMA:	01001
CME:	01010
CIR:	01011
CIL:	01100

INC:	01101
SPA:	01110
SNA:	01111
SZA:	10000
SZE:	10001
HLT:	10010
OUT:	10011
SKO:	10100
NOP:	10101

hex pad connect to microcontroller

https://www.circuitstoday.com/interfacing-hex-keypad-to-8051

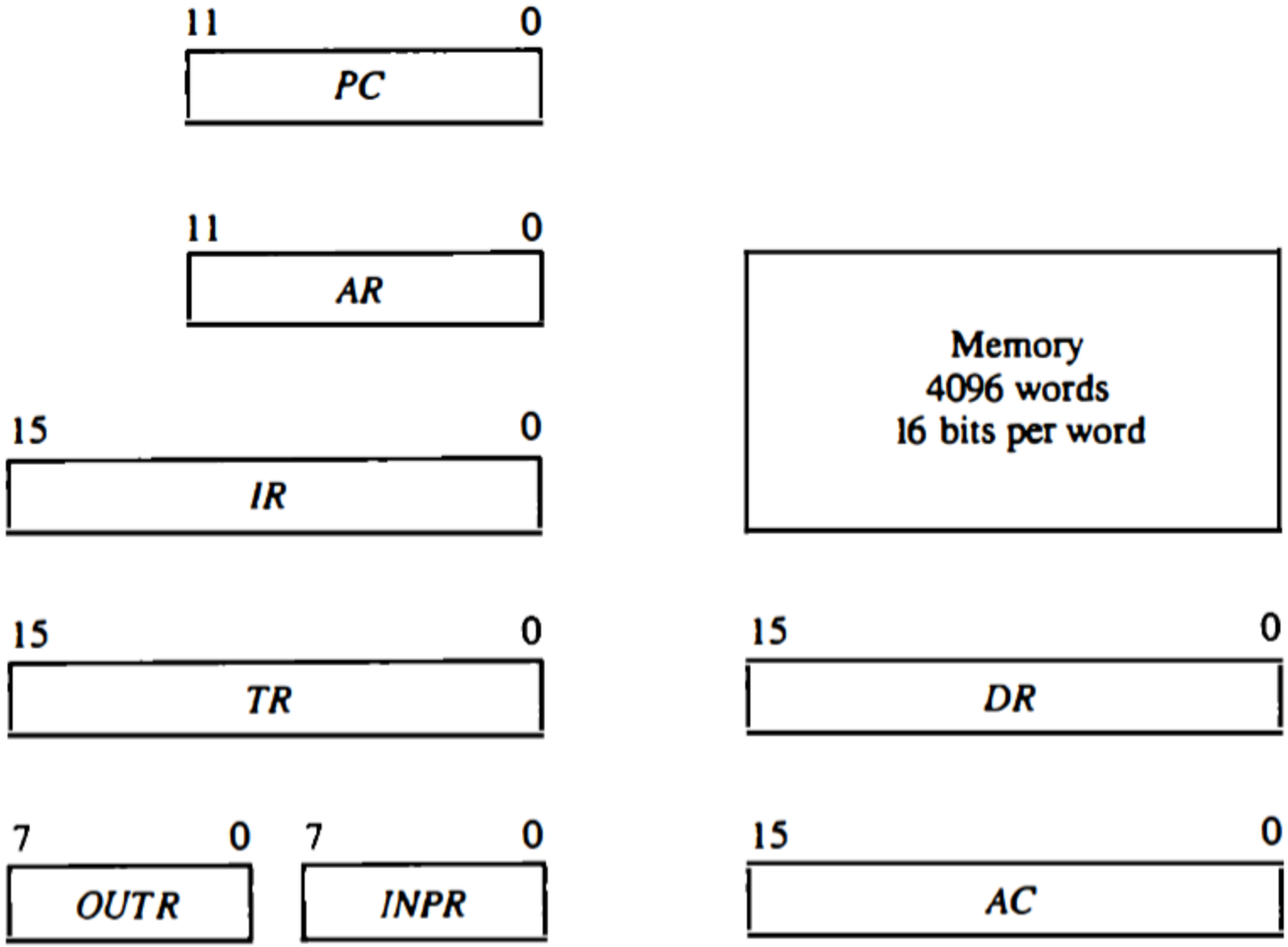
https://circuitdigest.com/microcontroller-projects/keypad-interfacing-with-avr-atmega32

Example

- https://github.com/yoosofan/mano-computer-simulator-js
- https://yoosofan.github.io/mano-computer-simulator-js/
- https://github.com/Naheel-Azawy/Simple-Computer-Simulator/blob/master/test/test-symbolic
- https://github.com/Naheel-Azawy/Simple-Computer-Simulator/blob/master/test/test
- https://github.com/Naheel-Azawy/Simple-Computer-Simulator/tree/master/test

Other assembly

- http://imrannazar.com/arm-opcode-map
- https://iitd-plos.github.io/col718/ref/arm-instructionset.pdf
- https://wiki.osdev.org/X86-64_Instruction_Encoding
- https://cs.brown.edu/courses/cs033/docs/guides/x64_cheatsheet.pdf
- https://sites.google.com/site/nttrungmtwiki/home/rce/assembly-language/x64-opcode-and-instruction-reference-home
- http://ref.x86asm.net/coder64.html
- arm 32 opcodes
- http://z80-heaven.wikidot.com/instructions-set:ld
- http://z80-heaven.wikidot.com/opcode-reference-chart
- https://smallcomputercentral.files.wordpress.com/2017/12/asm80-com-tutorial-e1-0-01.pdf
- https://stackoverflow.com/questions/22838444/convert-an-8bit-number-to-hex-in-z80-assembler
- https://www.vcfed.org/forum/forum/technical-support/vintage-computer-programming/76419-z80-hello-world-example-in-hex
- https://www.cemetech.net/forum/viewtopic.php?t=15710&start=0
- z80 assembly codes


11
0
PC
11
0
AR
15
0
IR
15
0
TR
7
0
OUTR
7
0
INPR
Memory
4096 words
16 bits per word
15
0
DR
15
0
AC

15	14 … 12	11 … 0
I	Opcode	Address

(Opcode = 000 through 110)

(a) Memory – reference instruction

15 … 12	11 … 0
0 1 1 1	Register operation

(Opcode = 111, $I = 0$)

(b) Register – reference instruction

15 … 12	11 … 0
1 1 1 1	I/0 operation

(Opcode = 111, $I = 1$)

(c) Input – output instruction

```
00101 00000  1010
00110 00000  1100
00111 00000  1110
01000

00000
```

اگر حداکثر ۳۲ دستور داشته باشیم پس پنج بیت برای دستورها نیاز داریم برای سادگی فرض می‌کنیم که طول همهٔ دستورها یکسان است یعنی هم دو بایت را می‌گیرند فرض کنید دستورها پنج بیت نیاز دارند پس ۱۱ بیت برای آدرس

حداکثر حافظهٔ این کامپیوتر چقدر می‌تواند باشد. اگر بخواهیم بایتی آدرس دهی کنیم

۲^۱۱ = ۲kB

B = Byte

اگر آدرس‌دهی را دو بایتی در نظر بگیریم

۴kB (word = 2 byte)

Output

LED

seven segment

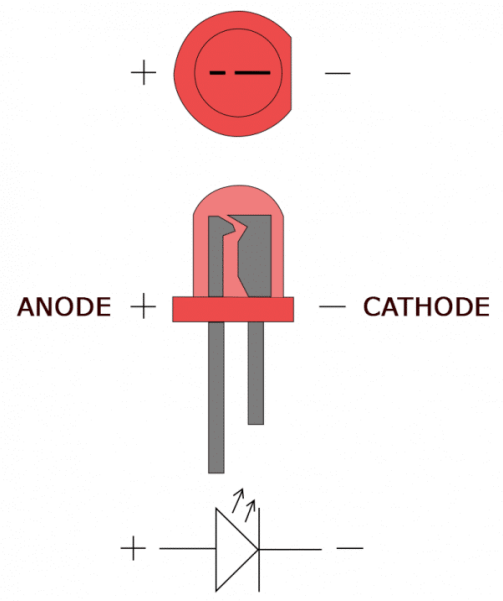
- https://www.rapidtables.com/convert/number/hex-to-binary.html
- convert hex to binary
- https://clrhome.org/asm/

Output Problem

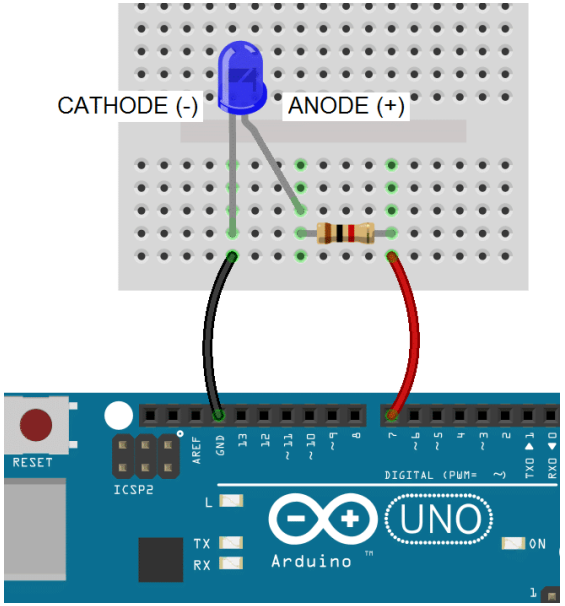
```
lda a
add b
sta c
out
hlt
a, 5
b, 2
c, 0
```

```
..........
..........

LB1: out
     sko
     bun LB1

...........
...........
```

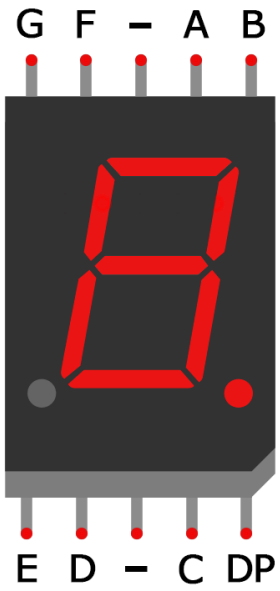
Simple LED

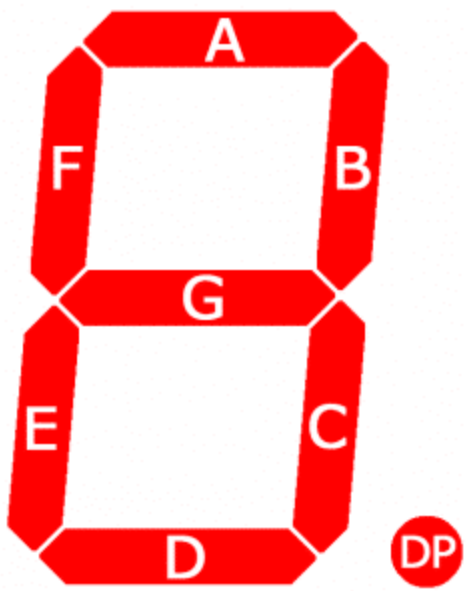


circuitbasics



Seven segment display





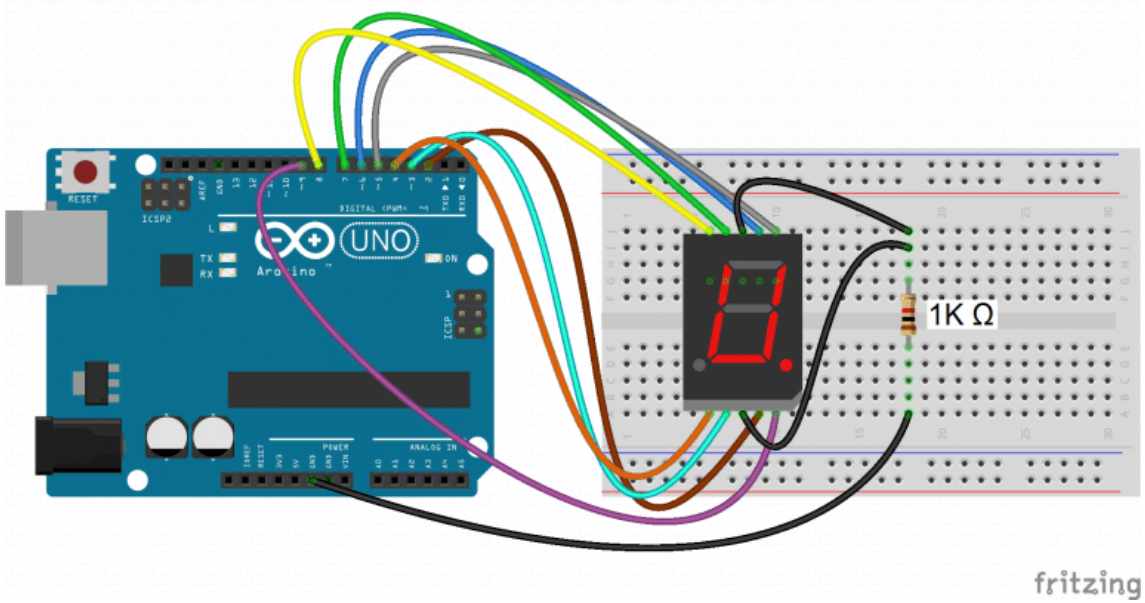
circuitbasics askingthelot

youtu.be element14

Imaginary Computer

- Consider it as real a computer
- Think about business plan
- Consider customers' need
- Consider other companies

YIC 30, Seven segment

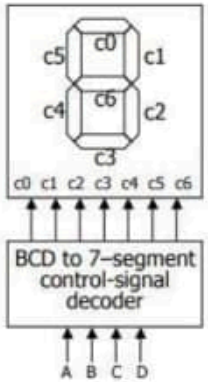


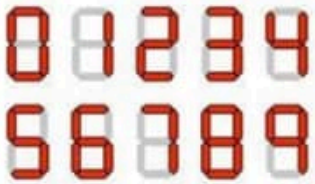
circuitbasics

Issues of YIC 30

- Convert binary number to 7 segment code
- Old codes only LED
- LED & seven segment
- Changing CPU
- Cost of changes
- Just one 7 segment ?
- for every output, seven segment code should be added

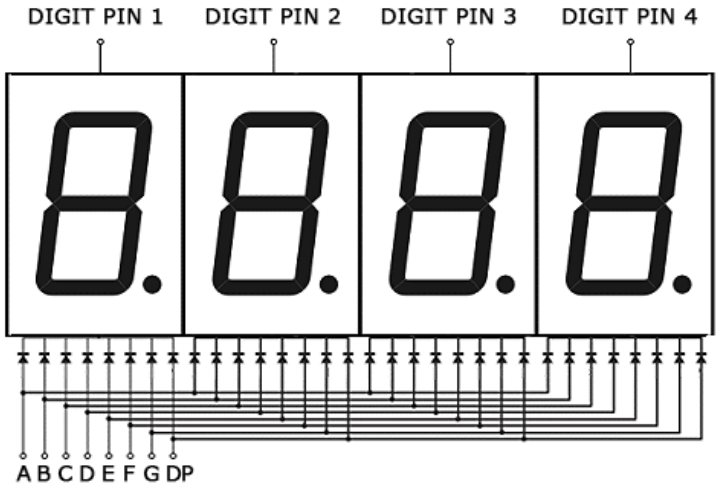
Hardware insead of Software

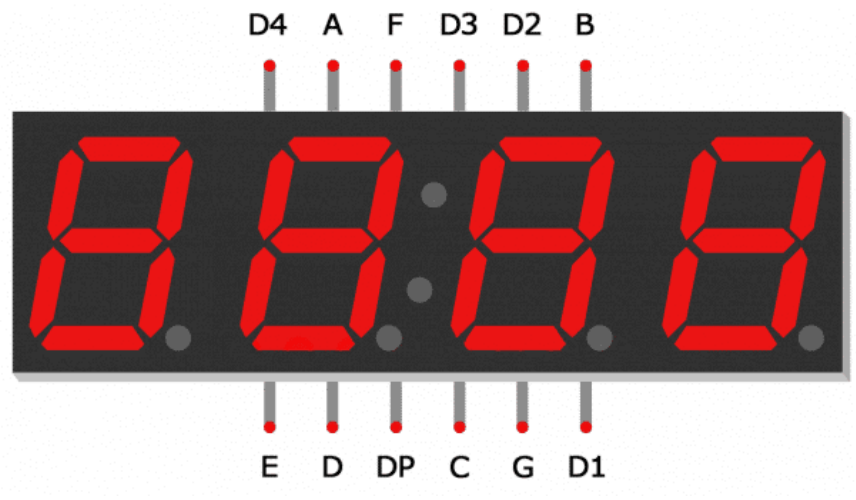




A	B	C	D	C0	C1	C2	C3	C4	C5	C6
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	0	0	1	1
1	0	1	–	–	–	–	–	–	–	–
1	1	–	–	–	–	–	–	–	–	–

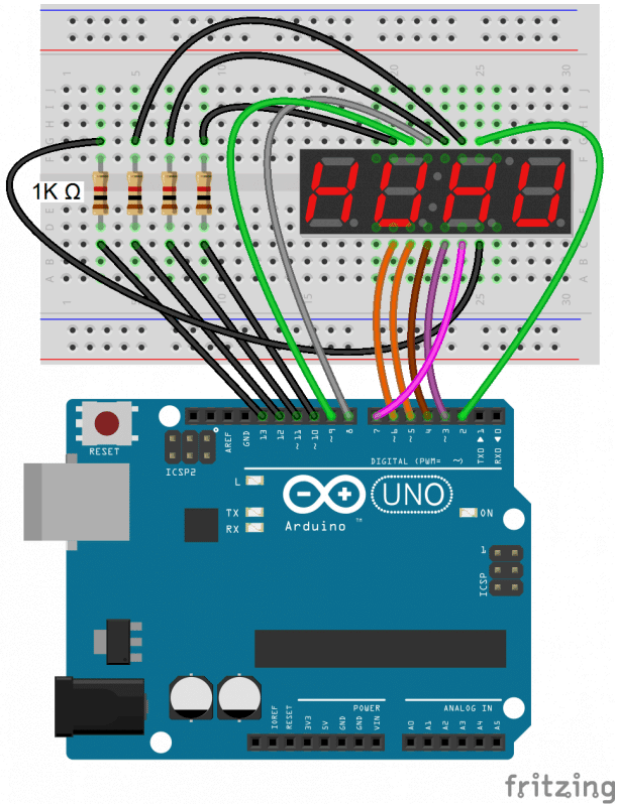
4 Digit 7-Segment Displays





circuitbasics

Connecting 4 Digit 7-Segment Displays



Arduino Print 4 to 7-segment

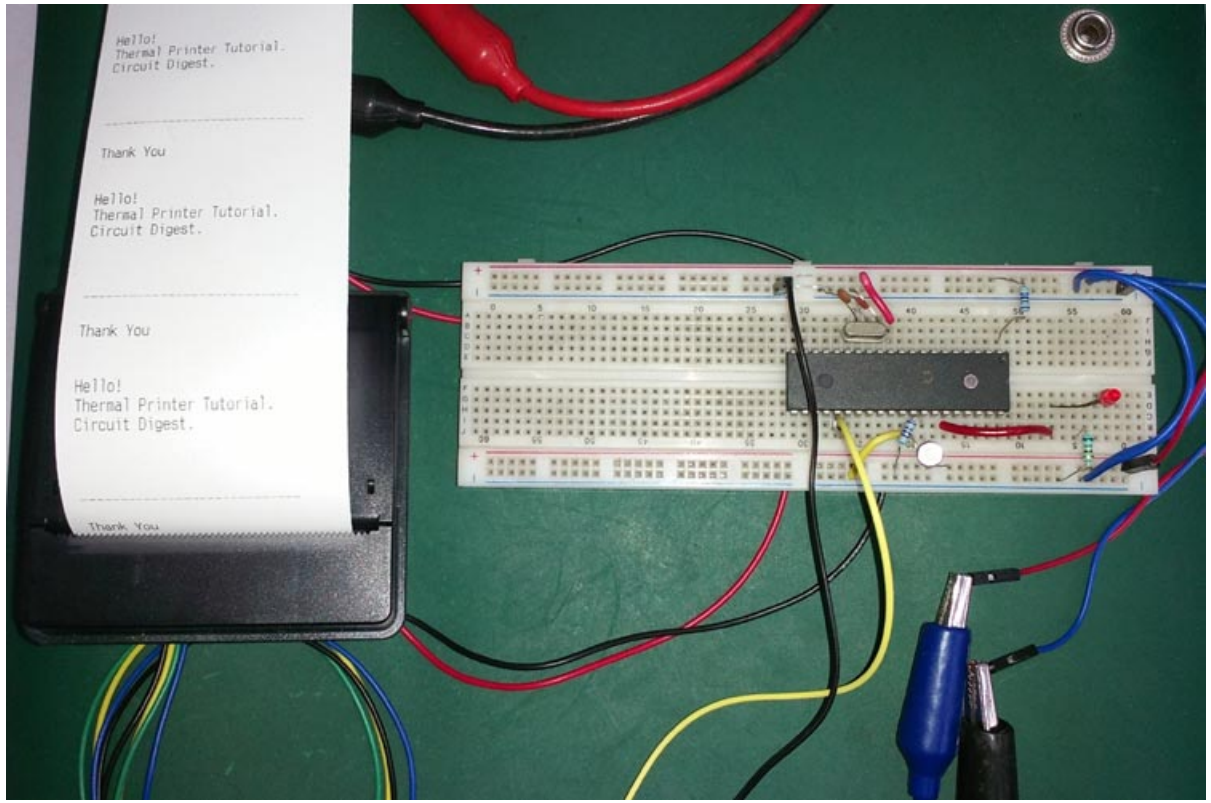
```
#include "SevSeg.h"
SevSeg sevseg;

void setup(){
  byte numDigits = 1;
  byte digitPins[] = {};
  byte segmentPins[] =
    {6, 5, 2, 3, 4, 7, 8, 9};
  bool resistorsOnSegments = true;

  byte hardwareConfig = COMMON_CATHODE;
  sevseg.begin(hardwareConfig,
    numDigits, digitPins, segmentPins,
    resistorsOnSegments
  );
  sevseg.setBrightness(90);
}

void loop(){
  sevseg.setNumber(4);
  sevseg.refreshDisplay();
}
```

Segment Pin	Arduino Pin
A	6
B	5
C	2
D	3
E	4
F	7
G	8
DP	9

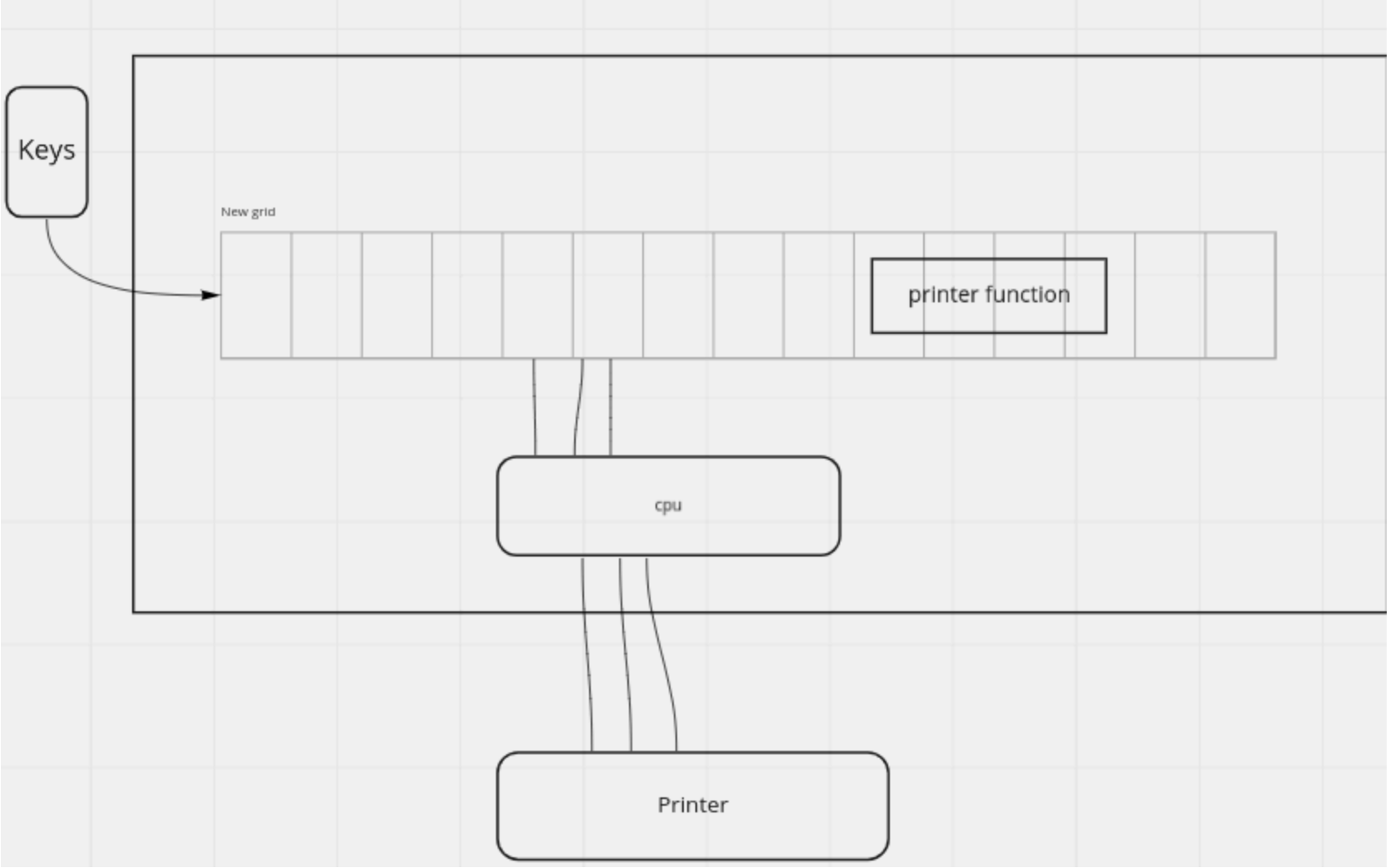
circuitbasics

YIC 40 - BSA

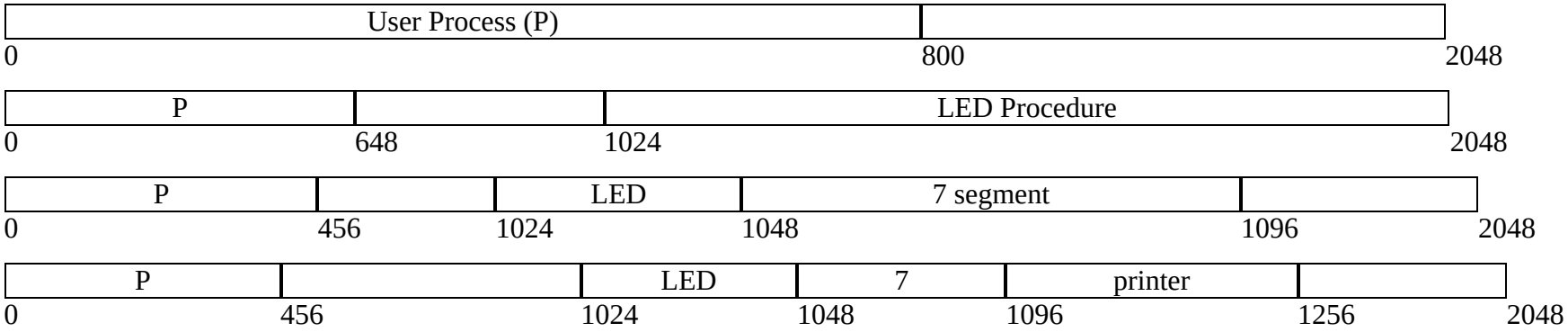
1. LED output code
2. 7 segment code
3. Printer
4. output selector
5. Adding porecedures
6. Device Drivers
7. Adding more devices
8. No error checking
9. Send data
10. API (protocol)



Printer Routine



Adding procedurs to memory



Users (programmers) should know where these precedures are

YIC50 - Array of Addresses

P		array	LED	7	printer	

0 456 1000 1024 1048 1096 1256 2048

Array

1024	1048	1096	1256	

0 1 2 3 4

indirect BSA

User chooses between old cpu or new one

YIC60 - Input Device (Card Reader)

1. Card Reader
2. Necessary Loops
3. Check Errors
4. Polling Method

- Hollerith and IBM keypunches, 1890
- IBM 011 Electric Key Punch(1923)
- IBM Type 032 Printing Punch(1935)
- A Key Punch Room in the 1960s

card punch

card reader

```
        ORG 0
START,  BSA READ1
        STA BYTE1
        LDA BYTE1
        BSZ OUTCH
        HLT

READ1,  HEX 0
RDCNT,  SKI
        BUN RDCNT
        INP
        BUN (READ1)

OUTCH,  HEX 0
OUTPR,  SKO
        BUN OUTPR
        OUT
        BUN (OUTCH)

BYTE1,  DEC     0
        END
```

Simple Input Output Code - YIC60

```
        ORG     0
START,  BSA     INPUT
        STA     NUM1

        BSA     INPUT
        STA     NUM2

        LDA     NUM1
        ADD     NUM2
        STA     RESULT

        BSA     OUTPUT

        HLT
```

```
INPUT,  HEX     0
POLL,   SKI
        BUN     POLL
        INP
        BUN     (INPUT)

OUTPUT, HEX     0
OUT_P,  SKO
        BUN     OUT_P
        OUT
        BUN     (OUTPUT)

// Data Section
NUM1,   DEC     0
NUM2,   DEC     0
RESULT, DEC     0

        END
```

YIC60 - Array of Addresses

P		array	LED	7	printer	

0 456 1000 1024 1048 1096 1256 2048

Array

1024	1048	1096	1256	

0 1 2 3 4

YIC60 code with array(I)

```
START    BSA     (PVT_INPUT)
         STA     NUM1
         BSA     (PVT_INPUT)
         STA     NUM2
         LDA     NUM1
         ADD     NUM2
         STA     RESULT
         BSA     (PVT_OUTPUT)
         HLT
NUM1,    DEC     0
NUM2,    DEC     0
RESULT,  DEC     0

         ORG     150
PVT_INPUT,  HEX 200
PVT_OUTPUT, HEX 220

         ORG     200
INPUT,   HEX     0
IN_POLL,SKI
         BUN     IN_POLL
         INP
         BUN     (INPUT)
```

```
         ORG     220
OUTPUT,  HEX     0
         STA     TEMP_OUT

OUT_POLL,SKO
         BUN     OUT_POLL
         LDA     TEMP_OUT
         OUT
         BSA     (PVT_DELAY)
         BUN     (OUTPUT)

         ORG     240
DELAY,   HEX     0
         LDA     DELAY_COUNT
         STA     DELAY_CTR

DELAY_LOOP,
         LDA     DELAY_CTR
```

YIC60 code with array(II)

```
DELAY,  HEX     0
        LDA     DELAY_COUNT
        STA     DELAY_CTR

DELAY_LOOP,
        LDA     DELAY_CTR
        SZA
        BUN     CONTINUE_DELAY
        BUN     (DELAY)

CONTINUE_DELAY,
        DEC
        STA     DELAY_CTR
        BUN     DELAY_LOOP

        ORG     300
TEMP_OUT,   DEC     0
DELAY_COUNT,DEC     10
```

```
DELAY_CTR,  DEC     0
PVT_DELAY,  HEX     52
        END
```

YIC70 - Adding Loader

Loader	user process		array	LED	7	printer	Card Reader	

0 100 556 1000 1024 1048 1096 1256 1450 2048

Array

1024	1048	1096	1256	

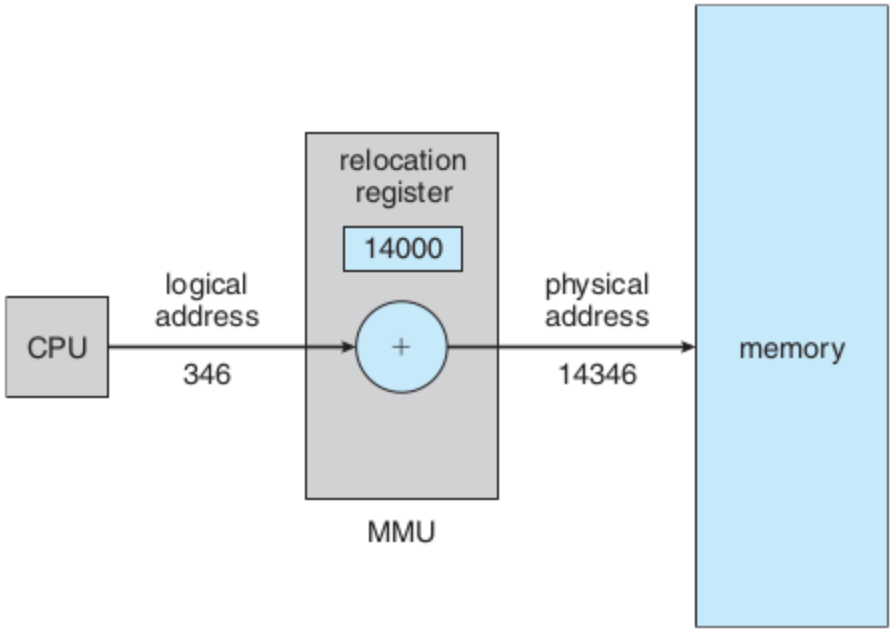
0 1 2 3 4

```
        ORG     0
START,  BSA     READ_COUNT
        STA     BYTE_COUNT
        BSA     INIT_LOAD
        BSA     LOAD_LOOP
        BSA     EXECUTE
        BUN     0
        HLT

READ_COUNT, HEX 0
RD_CNT, SKI
        BUN     RD_CNT
        INP
        BUN     READ_COUNT I

INIT_LOAD, HEX 0
        LDA     ZERO
        STA     LOAD_PTR
        STA     CURRENT_IDX
        BUN     INIT_LOAD I
```

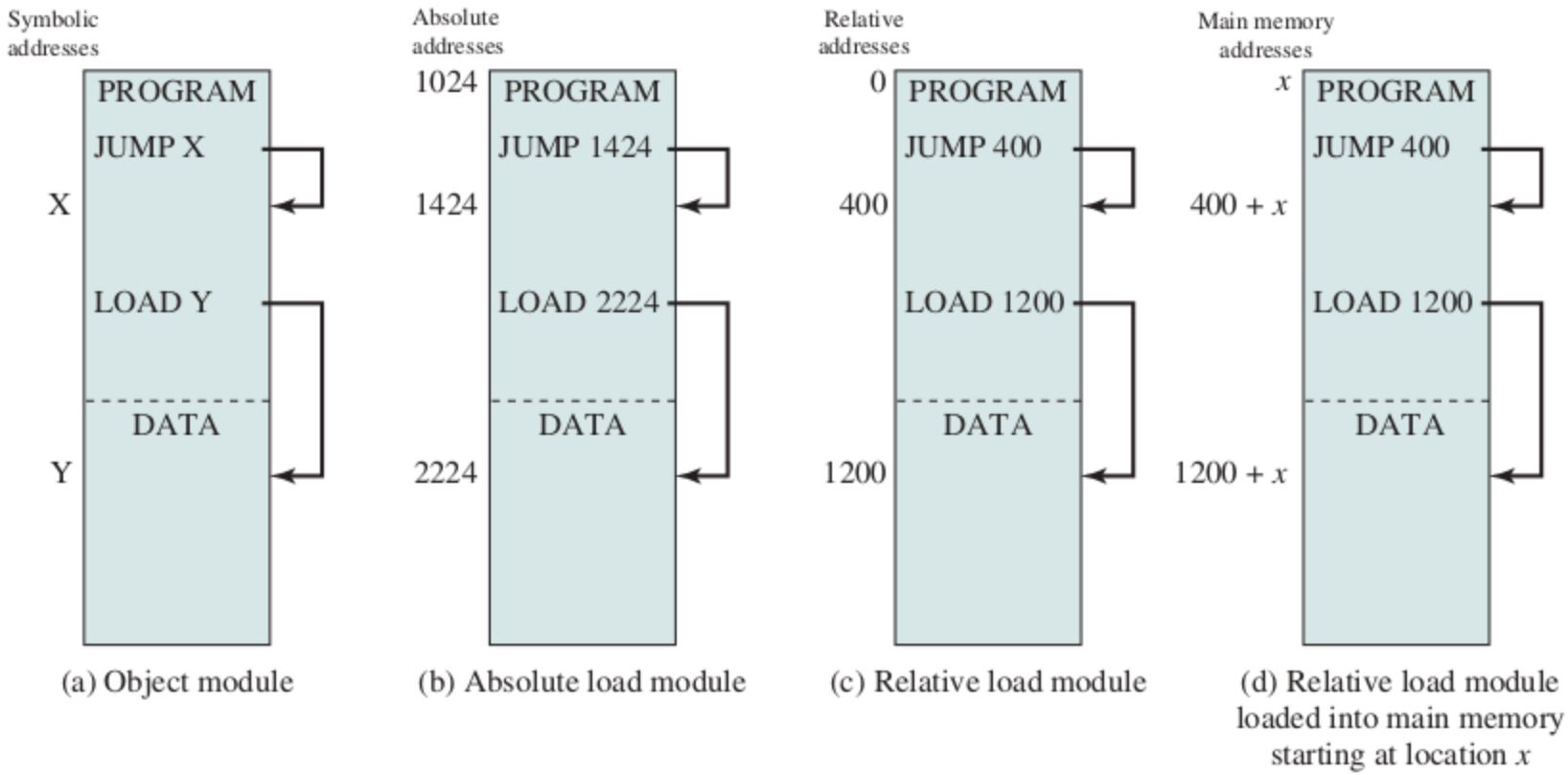
```
LOAD_LOOP, HEX 0
        LDA     CURRENT_IDX
        SUB     BYTE_COUNT
        SPA
        BUN     LOAD_BYTE
        BUN     LOAD_LOOP I

LOAD_BYTE,
        BSA     READ_BYTE
        STA     TEMP_BYTE

        LDA     LOAD_PTR
        STA     STORE_PTR
        LDA     TEMP_BYTE
        STA     STORE_PTR I

        LDA     LOAD_PTR
        INC
        STA     LOAD_PTR
        LDA     CURRENT_IDX
        INC
        STA     CURRENT_IDX
        BUN     LOAD_LOOP
```

```
READ_BYTE, HEX 0
RD_BYT, SKI
        BUN     RD_BYT
        INP
        BUN     READ_BYTE I

        ORG     128
EXECUTE, HEX 0

        LDA     ZERO
        STA     RESULT
        BUN     EXECUTE I

        ORG     64
ZERO,       DEC     0
BYTE_COUNT, DEC     0
LOAD_PTR,   HEX     128
CURRENT_IDX,DEC     0
TEMP_BYTE,  DEC     0
STORE_PTR,  HEX     0
RESULT,     DEC     0
        END
```

YIC75 Relative Address

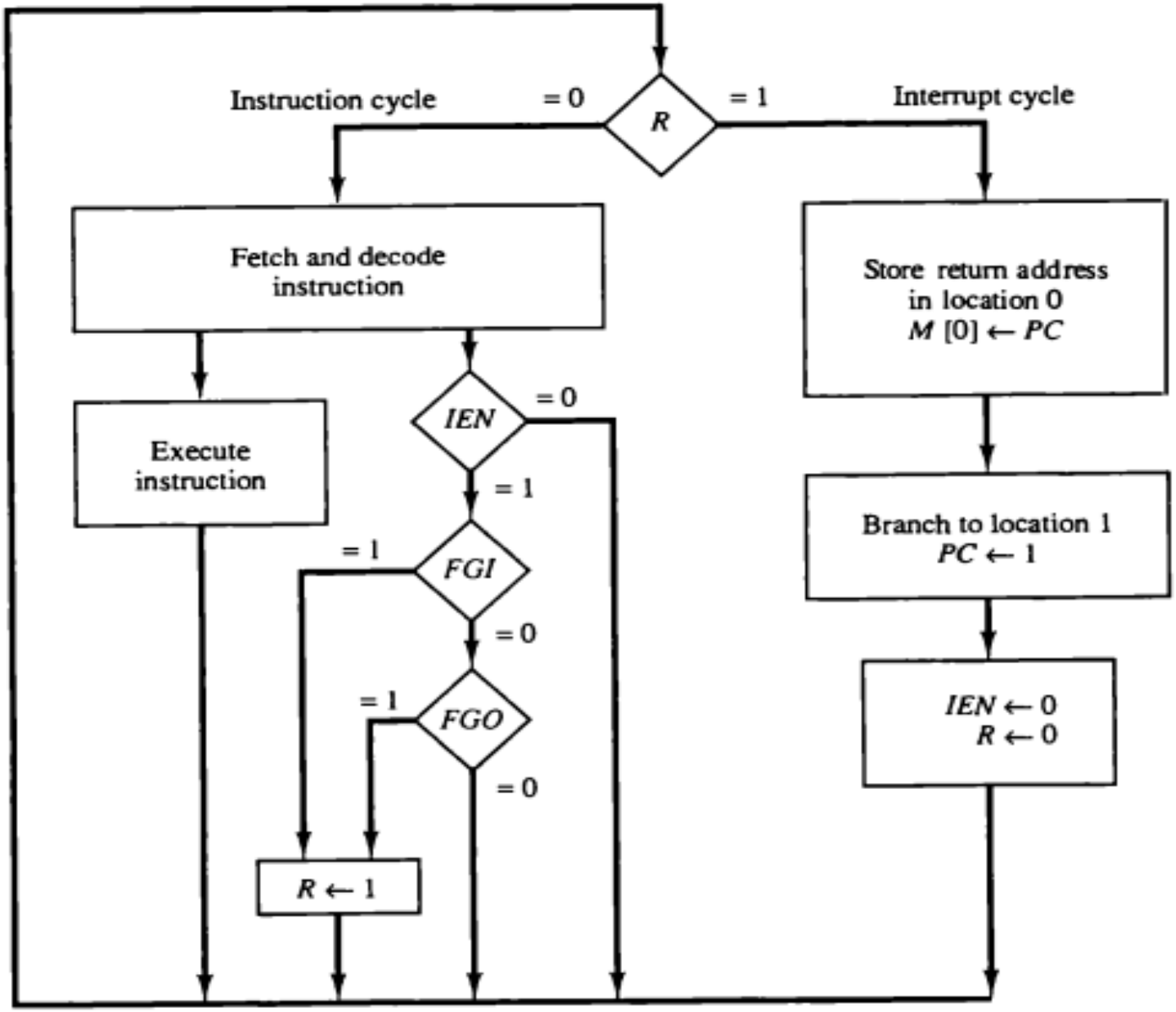


address binding, absolute and relocate loader



(a) Object module (b) Absolute load module (c) Relative load module (d) Relative load module loaded into main memory starting at location x

YIC80 - Interrupt



Interrupt-Driven Program

```
        ORG     0
        BUN     MAIN
        BUN     ISR

MAIN,   LDA     ZERO
        STA     COUNT
        ION

WORK,   LDA     COUNT
        INC
        STA     COUNT
        BUN     WORK

ISR,    STA     SAVE
        BSA     IO
        LDA     SAVE
        ION
        BUN     0  I
```

Save E flag

```
IO,     HEX     0
        SKI
        BUN     OUTPUT
        INP
        STA     BUFFER
        BUN     IO  I
OUTPUT, SKO
        BUN     IO  I
        OUT
        BUN     IO  I

ZERO,   DEC     0
COUNT,  DEC     0
SAVE,   DEC     0
BUFFER, DEC     0
        END
```

Input, Add and Output

```
        ORG     0
        BUN     MAIN
        BUN     ISR
MAIN,   LDA     ZERO
        STA     READY
        ION
WAIT,   LDA     READY
        SZA
        BUN     PROCESS
        BUN     WAIT
PROCESS,LDA     BUFFER
        ADD     TEN
        STA     BUFFER
        BSA     OUT_SUB
        HLT

OUT_SUB,HEX     0
POLL_O, SKO
        BUN     POLL_O
        LDA     BUFFER
        OUT
        BUN     OUT_SUB I

ISR,    BSA     SAVE_SUB
        SKI
        BUN     ISR_EXIT

        INP
        STA     BUFFER
        LDA     ONE
        STA     READY

ISR_EXIT,BSA     RSTR_SUB
        ION
        BUN     0 I

SAVE_SUB,HEX    0
        STA     TEMP_AC
        CIL
        STA     TEMP_E
        BUN     SAVE_SUB I

RSTR_SUB,HEX    0
        LDA     TEMP_E
        CIR
        LDA     TEMP_AC
        BUN     RSTR_SUB I

ZERO,   DEC     0
ONE,    DEC     1
TEN,    DEC     10
READY,  DEC     0
BUFFER, DEC     0
TEMP_AC,DEC     0
TEMP_E, DEC     0
        END
```

	Loader	READ	WRITE	ISR	DATA	user process

0 4 100 150 200 250 300 1024

```
    BUN     LOADER_MAIN
    BUN     ISR
LOADER_MAIN, ION
LOAD_LOOP,BSA  READ_WORD
    STA     PROG_SIZE
    BSA     LOAD_PROGRAM
    BSA     PROGRAM_BASE
    BUN     LOAD_LOOP
LOAD_PROGRAM, HEX 0
    LDA     ZERO
    STA     LOAD_INDEX
LOAD_PROGRAM_LOOP,
    LDA LOAD_INDEX
    SUB     PROG_SIZE
    SPA
    BUN     LOAD_DONE
    BSA     READ_WORD
    STA     TEMP_DATA
    LDA     LOAD_INDEX
    ADD     PROGRAM_BASE
    STA     TEMP_ADDR
    LDA     TEMP_DATA
    STA     TEMP_ADDR I
    LDA     LOAD_INDEX
    INC
    STA     LOAD_INDEX
    BUN     LOAD_PROGRAM_LOOP
LOAD_DONE, BUN LOAD_PROGRAM I
    ORG     100
READ_WORD, HEX 0
WAIT_FOR_INPUT, LDA INPUT_READY
    SZA
    BUN     HAVE_INPUT
    BUN     WAIT_FOR_INPUT
HAVE_INPUT,
    LDA     INPUT_BUFFER
    STA     TEMP_DATA
    LDA     ZERO
    STA     INPUT_READY
    LDA     TEMP_DATA
    BUN     READ_WORD I
    ORG     150
WRITE_WORD, HEX 0
    STA     OUTPUT_DATA
    LDA     ONE
    STA     OUTPUT_PENDING
WAIT_FOR_OUTPUT,
    LDA OUTPUT_PENDING
    SZA
    BUN     WAIT_FOR_OUTPUT
    BUN     WRITE_WORD I
    ORG     200
ISR,    STA    SAVE_AC
    SKI
    BUN     CHECK_OUTPUT
    INP
    STA     INPUT_BUFFER
    LDA     ONE
    STA     INPUT_READY
    BUN     ISR_EXIT
CHECK_OUTPUT, SKO
    BUN     ISR_EXIT
    LDA     OUTPUT_PENDING
    SZA
    BUN     PERFORM_OUTPUT
    BUN     ISR_EXIT
PERFORM_OUTPUT,
    LDA OUTPUT_DATA
    OUT
    LDA     ZERO
    STA     OUTPUT_PENDING
ISR_EXIT, LDA   SAVE_AC
    BUN     0 I
    ORG     250
ZERO,           DEC 0
ONE,            DEC 1
PROG_SIZE,      DEC 0
LOAD_INDEX,     DEC 0
TEMP_ADDR,      HEX 0
TEMP_DATA,      DEC 0
INPUT_BUFFER,   DEC 0
INPUT_READY,    DEC 0
OUTPUT_DATA,    DEC 0
OUTPUT_PENDING, DEC 0
SAVE_AC,        DEC 0
PROGRAM_BASE,   HEX 300
        END

  ORG 300 ; User program
USER_ENTRY, HEX 0
  BSA 100
  ADD TEN
  BSA 150
  BUN USER_ENTRY I
  TEN, DEC 10
  END
```

Other Loader Samples

```
# Advanced: Buffered Input with BSA Subroutines
src/in/Interrupt_Driven_Program_with_BSA_Subroutines_Advanced_with_buffer.asm
src/in/Interrupt_Driven_Program_with_BSA_Subroutines_Advanced_with_buffer_comments.asm

# Enhanced bootstrap loader with error checking
src/in/Bootstrap_Loader_Program_More_Robust_Version_with_Error_Checking.asm

# Relocating Bootstrap Loader with Base Register
src/in/loader4_base_register_comments.asm


# Simple Bootstrap Loader with Absolute Addressing
src/in/loader7_interrupt_Simple_Bootstrap_Loader_with_Absolute_Addressing.asm
src/in/loader7_interrupt_Simple_Bootstrap_Loader_with_Absolute_Addressing_comments.asm

src/in/loader7_user_program_comments.asm
src/in/loader7_user_program.asm


# Uses interrupt-driven I/O instead of polling
src/in/loader10_interrupt.asm
src/in/loader10_interrupt_comments.asm

# Interrupt-driven program that gets loaded
src/in/loader10_loaded_program.asm
src/in/loader10_loaded_program_comments.asm
```

Review YICs

1. Connect memory to power
2. Connect HEX Pad keyboard to memory.
3. Enter machine code of user
4. Enter machine code of OS Routines
5. Connect cpu to memory
6. Connect output device to cpu
7. Connect input device to cpu
8. Turn on CPU (start runnig codes)
9. HLT at the end of user code
10. Give output(s) to user
11. Back to step 1

Relative Address Problem

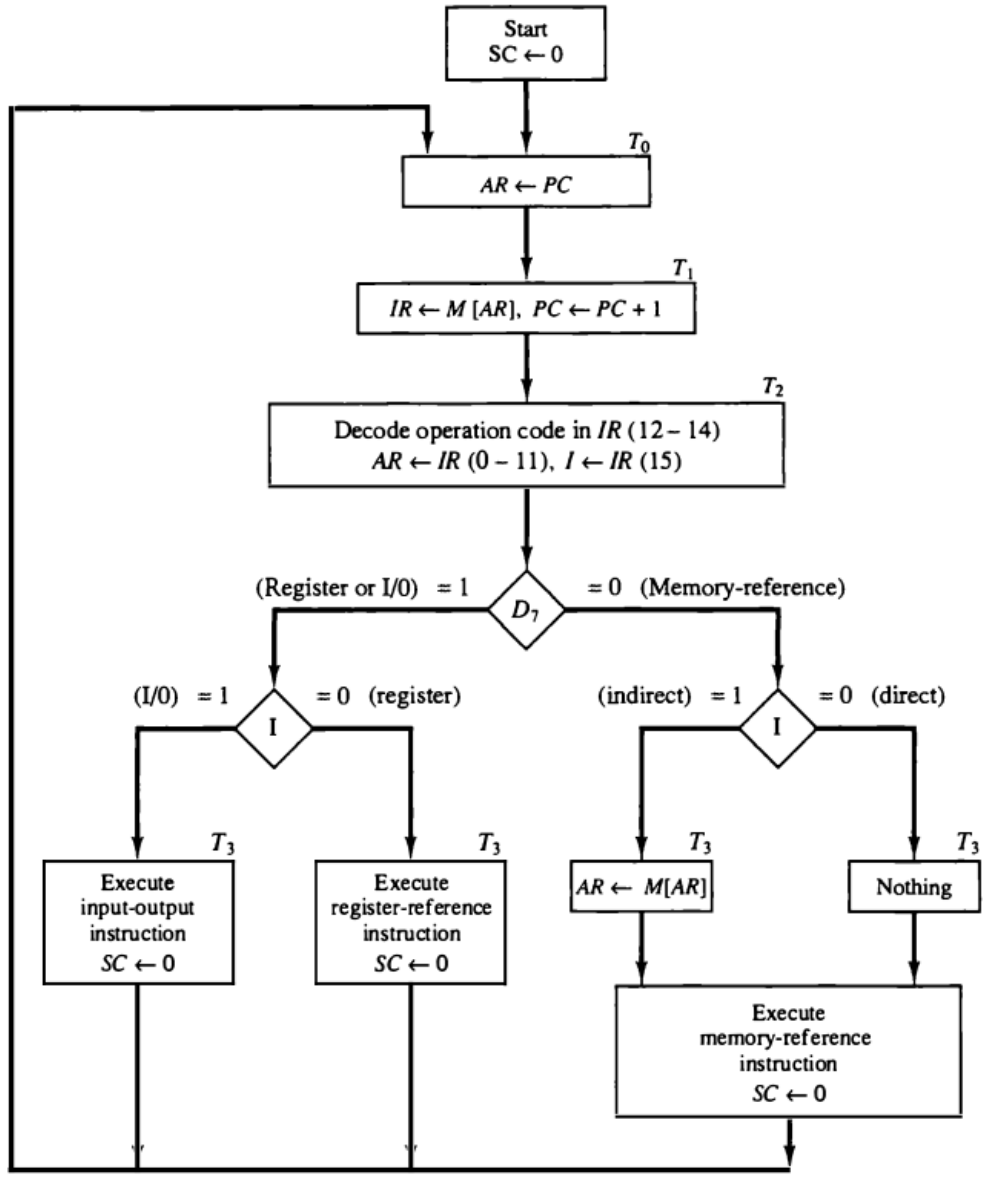
- Normal execution
    - Logical Address
    - Base or Relocation Register
    - Physical address ← PC + Base
    - AR ← Base + PC
    - Kernel Mode
    - UMD: Change CPU mode to user
    - mode = 1, a flag in CPU Like C
- Kernel
    - Interrupt Service Routine (ISR)
    - Boot
    - Loader
    - OS routines and libraries
    - Effective address = logical address
    - KMD: change CPU mode to kernel
    - mode = 0

There is a flaw in your code. In YIC 80 and 90, the kernel (loader, ISR and other system routines) put in the memory (RAM, random access memory) directly by a device like HEX Pad(Hexadecimal) keyboard without CPU involvement.

1. Turn on memory by connecting to power supply (battery or anything like that) and will not turn off afterwards
2. Connect HEX Pad keyboard (or any similar device) to memory.
3. Enter kernel machine code by the Hexpad keyboard
4. Disconnect HEX Pad keyboard from memory after kernel entered to the memory
5. Connect cpu to memory
6. Connect printer or another simple output device to cpu
7. Connect card reader or another simple input device to cpu
8. Turn on cpu by connecting it to a power supply (battery or anything like that)
9. Mano's cpu starts at 0 address
10. Read user program from input device by loader (part of kernel)
11. RTI to user program
12. RTK to loader after the user program finished
13. Load another user program from input

Your code put a hex 0 in address zero of memory while it should be a BUN Please fix this problem and any other problems in your suggested code and rewrite the whole code.

```
ORG 0
BUN     LOADER
BUN     ISR
LOADER, ION
LOOP,   BSA READ_WORD
STA     SIZE
BSA     LOAD
UMD
BSA     300
BUN     LOOP

; ISR
; Routines
END

ORG     300 ; User program
UPRG,   HEX 0
BSA     100
ADD     TEN
BSA     150
BUN     300 I
TEN,    DEC 10
END
```


Start
SC ← 0
T0
AR ← PC
T1
IR ← M [AR], PC ← PC + 1
T2
Decode operation code in IR (12 – 14)
AR ← IR (0 – 11), I ← IR (15)
(Register or I/0) = 1
D7
= 0 (Memory-reference)
(I/0) = 1
I
= 0 (register)
(indirect) = 1
I
= 0 (direct)
T3
Execute
input-output
instruction
SC ← 0
T3
Execute
register-reference
instruction
SC ← 0
T3
AR ← M[AR]
T3
Nothing
Execute
memory-reference
instruction
SC ← 0

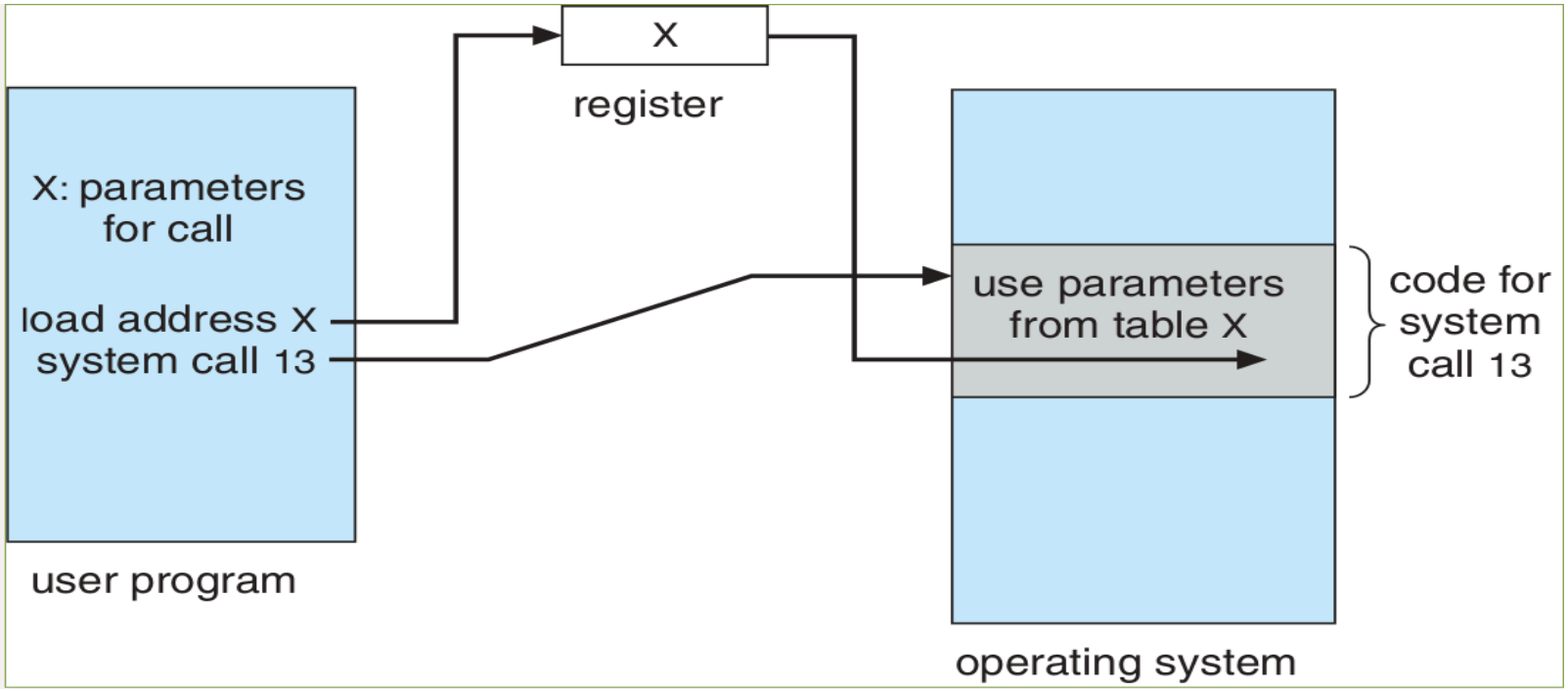
YIC90 Interrupt and Relative Address

RTI instruction

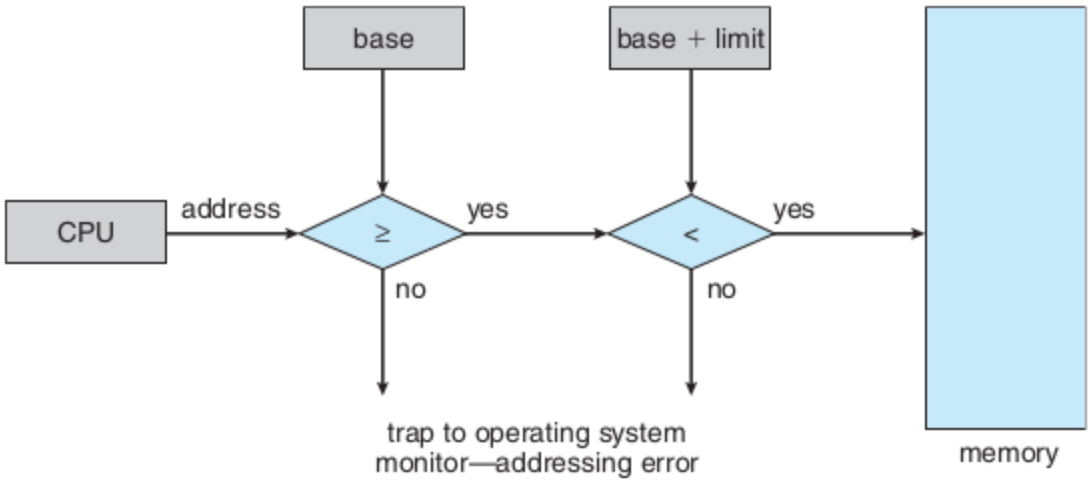
When the CPU executes RTI (while still safely in Kernel Mode), the hardware performs the following sequence during the execution phase:

1. **t_3: $AR leftarrow 0$** *(Put absolute address 0 into the Address Register)*
2. **t_4: $DR leftarrow M[AR]$** *(Read the saved return address from memory into the Data Register)*
3. **t_5: $PC leftarrow DR$, $MODE leftarrow 1$, $IEN leftarrow 1$, $SC leftarrow 0$** *(Simultaneously restore the Program Counter, switch the Mode flip-flop to User, re-enable hardware interrupts, and clear the Sequence Counter to end the instruction)*

Simple Parameters



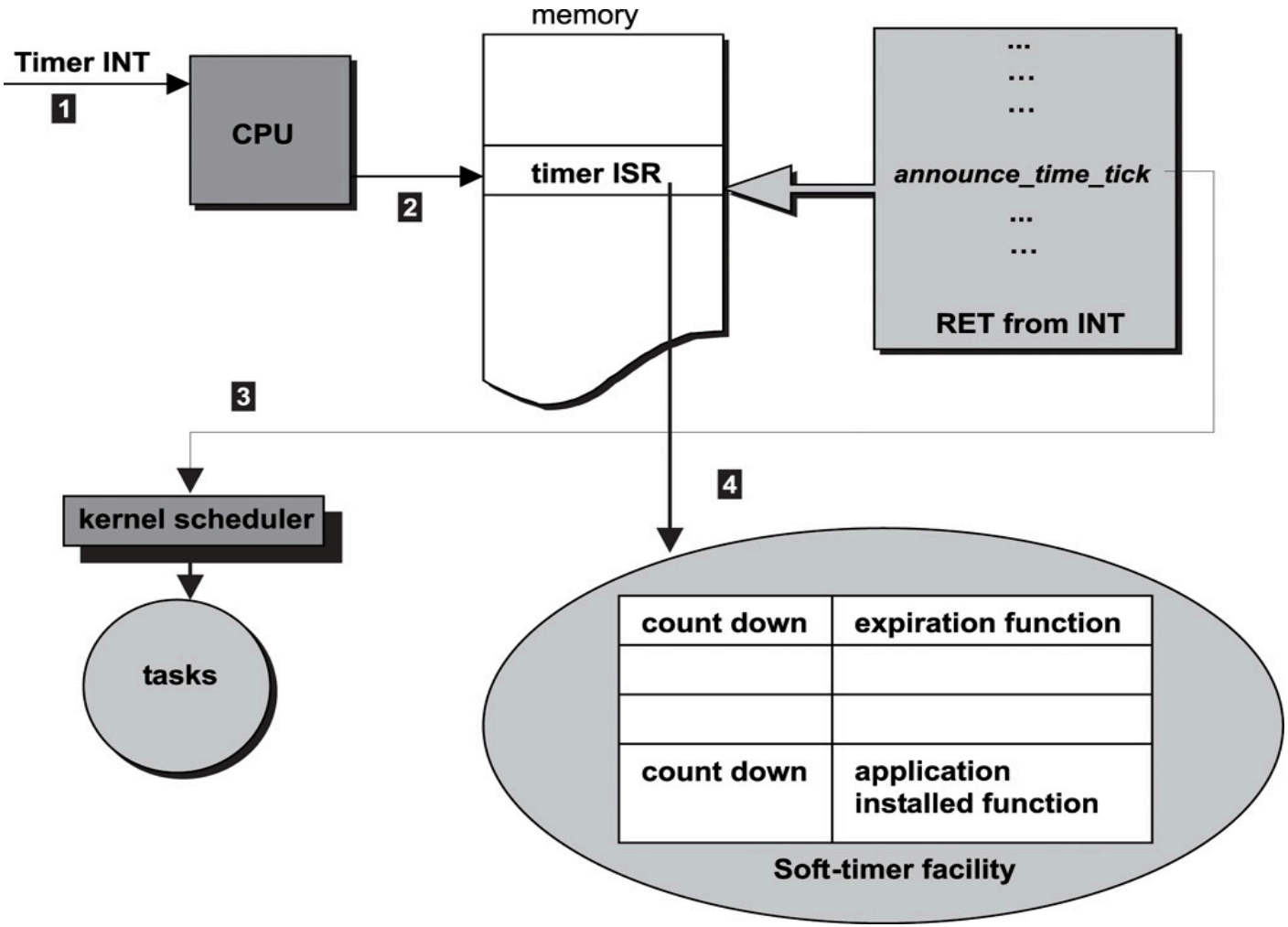
YIC 100 Memory Protection



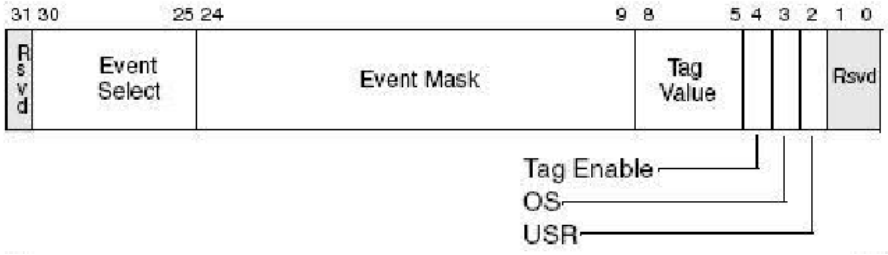
1. SKT like SKI and SKO
2. System Call ?
3. Change registers by the running process

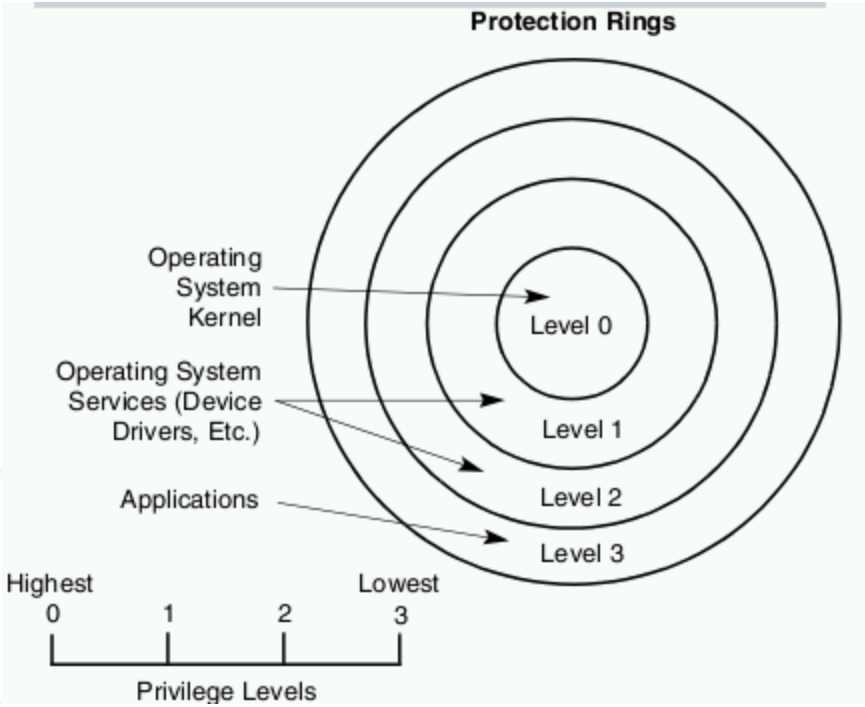
CPU protection

Timer interrupt



YIC 110 cpu protection



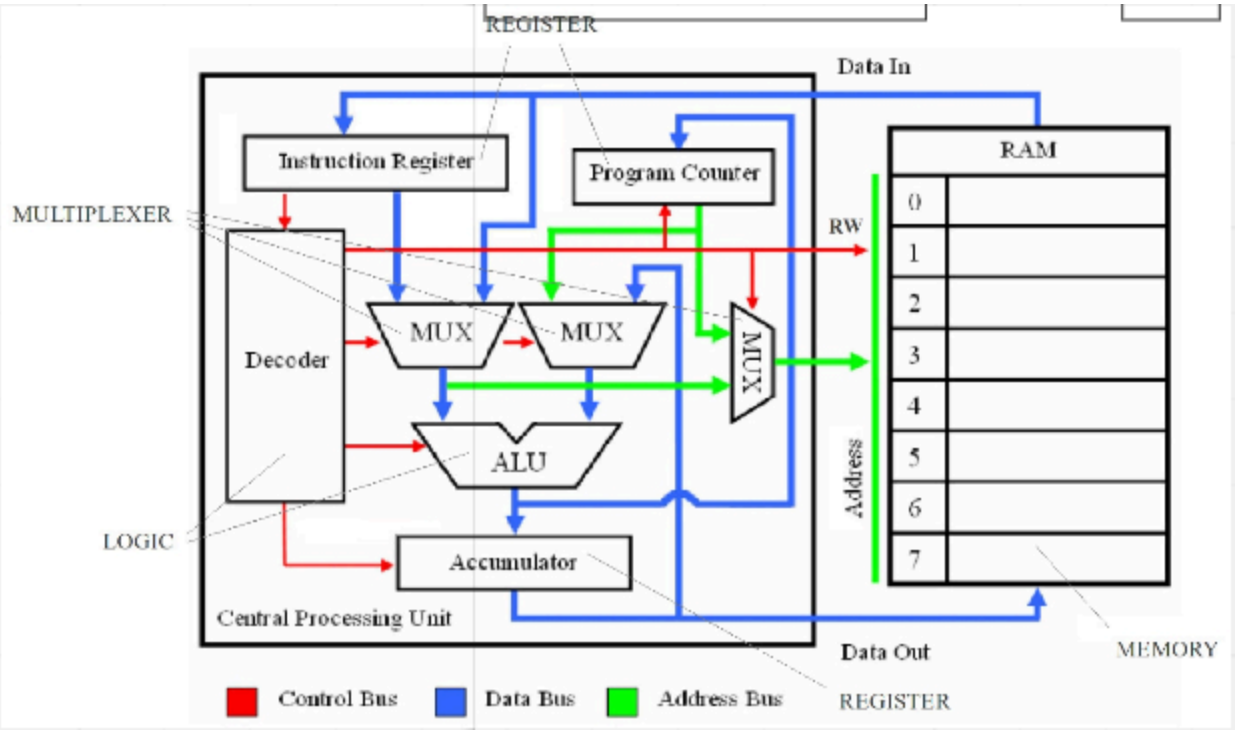


Function call

- cons of BSA
    - No recursion
    - No explicit data transfer

Stack From end

Call, Ret

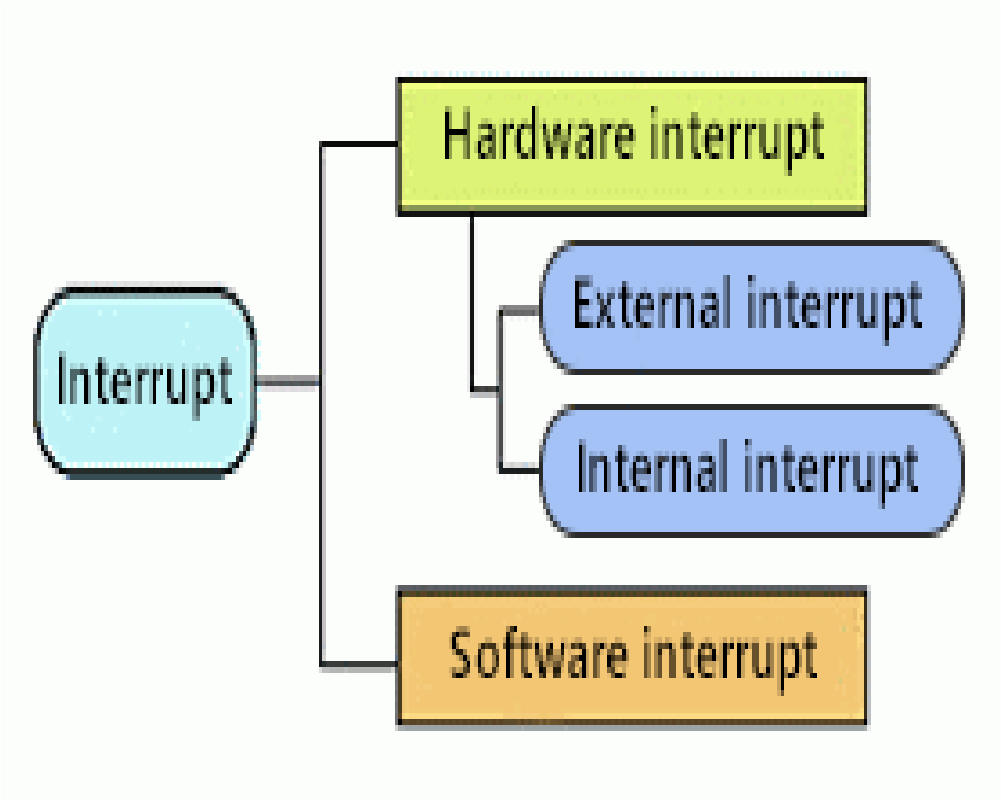


- [Assembly Slides]

Software Interrupt

```
ISR,     STA     SAVE
         BSA     IO
         LDA     SAVE
         ION
         BUN     0  I

IO,      HEX     0
         SKI
         BUN     OUTPUT
         INP
         STA     BUFFER
         BUN     IO  I
OUTPUT,  SKO
         BUN     TRAP
         OUT
TRAP,    SKT
         BUN     IO  I
         BUN     100
```

```
mov ah, 0x0e

; function number = 0Eh
; : Display Character

mov al, '!'

; AL = code of character
; to display

int 0x10

; call INT 10h,
; BIOS video service
```


Interrupt
Hardware interrupt
External interrupt
Internal interrupt
Software interrupt

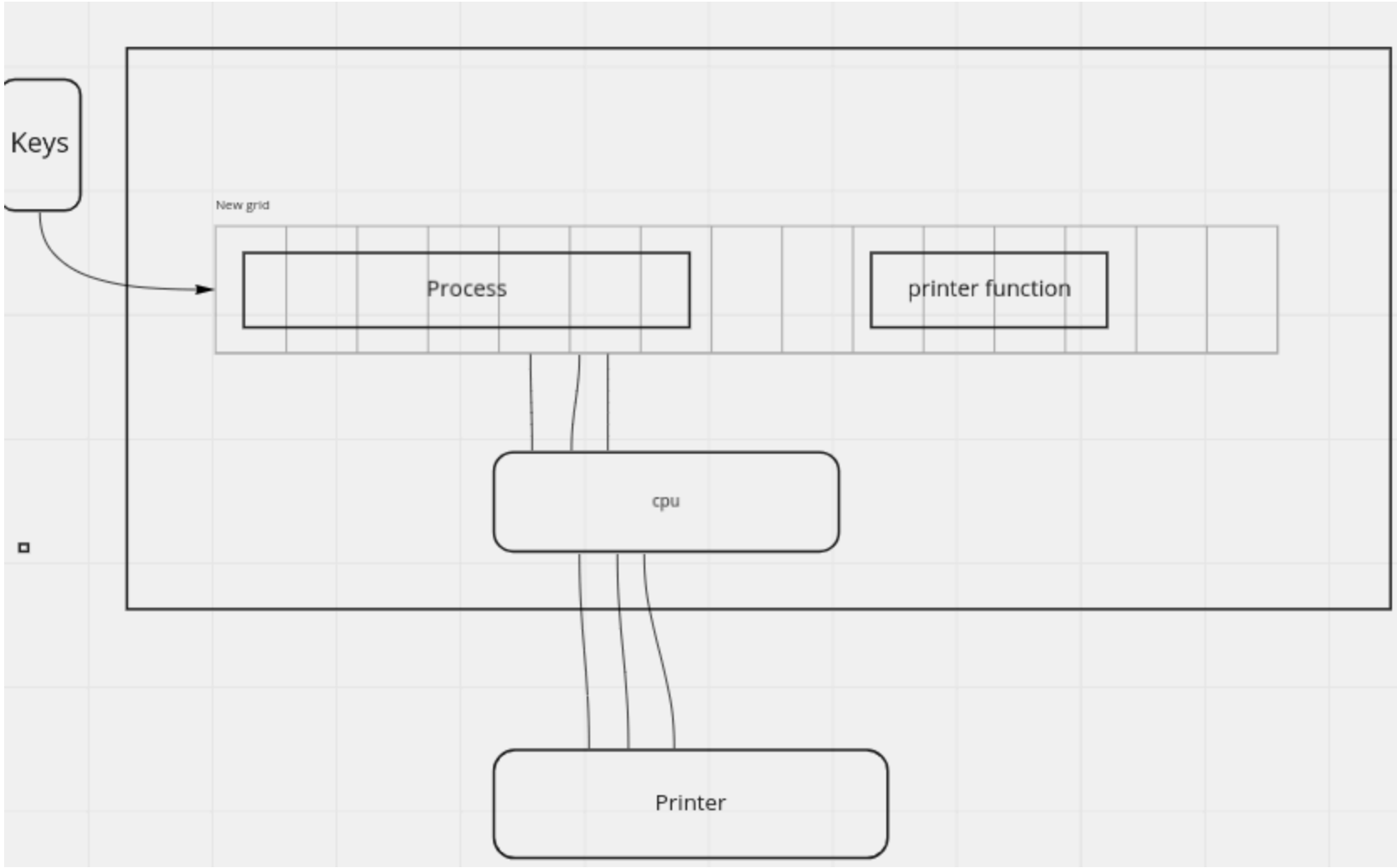
Adding PSW

Flags:

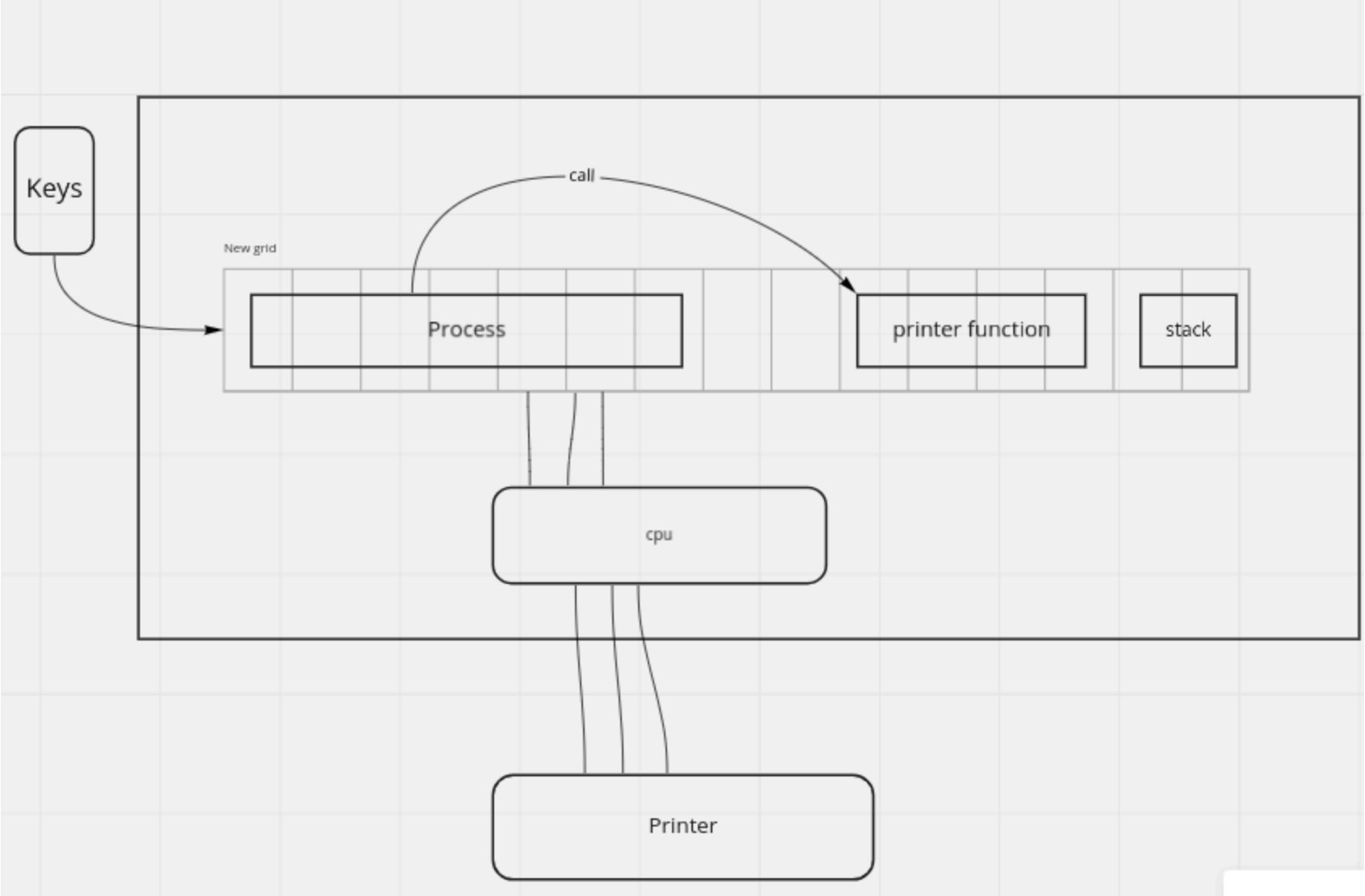
- Carry
- E
- Mode

YIC 150 - Multiprogramming

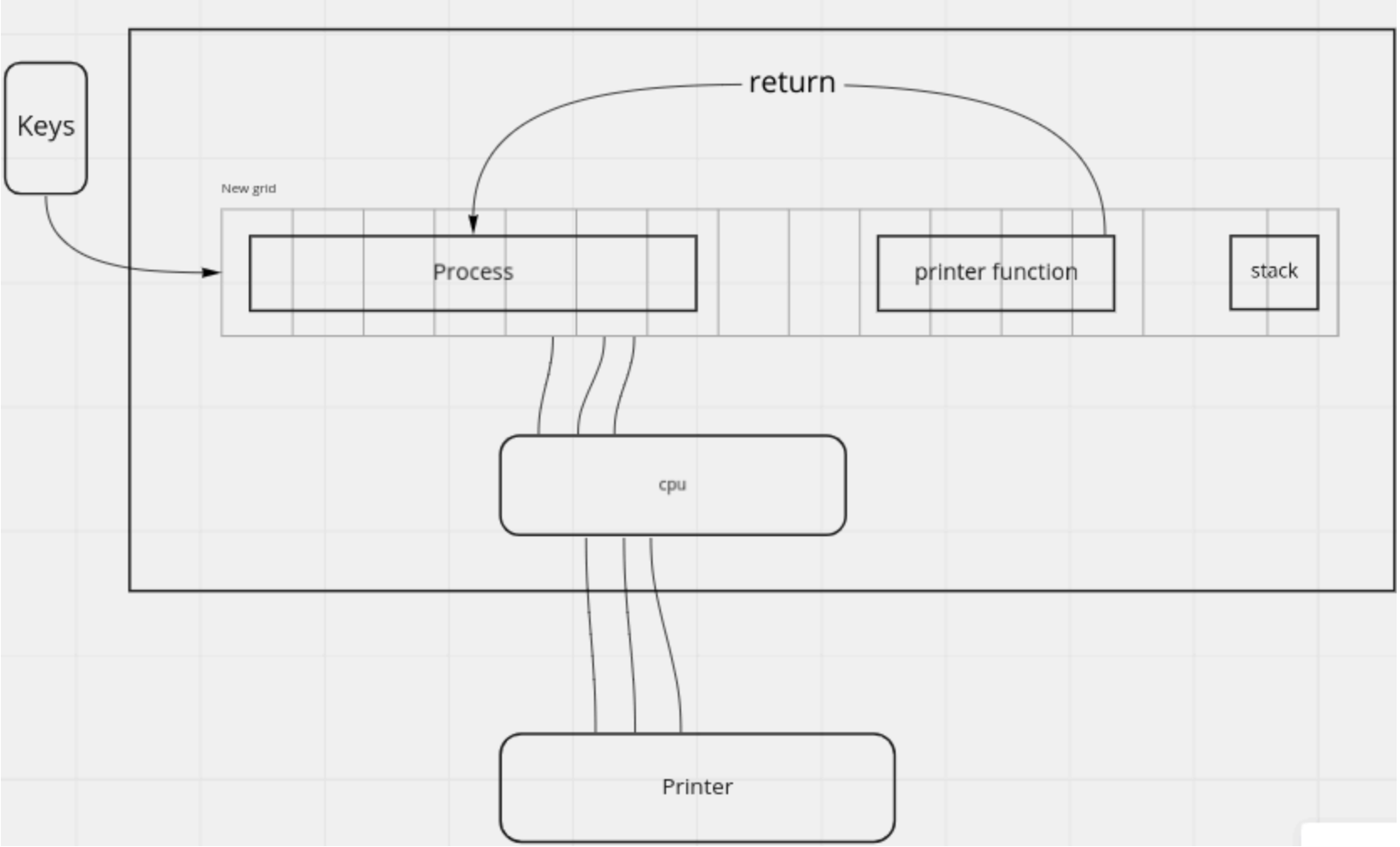
Process along Printer Function



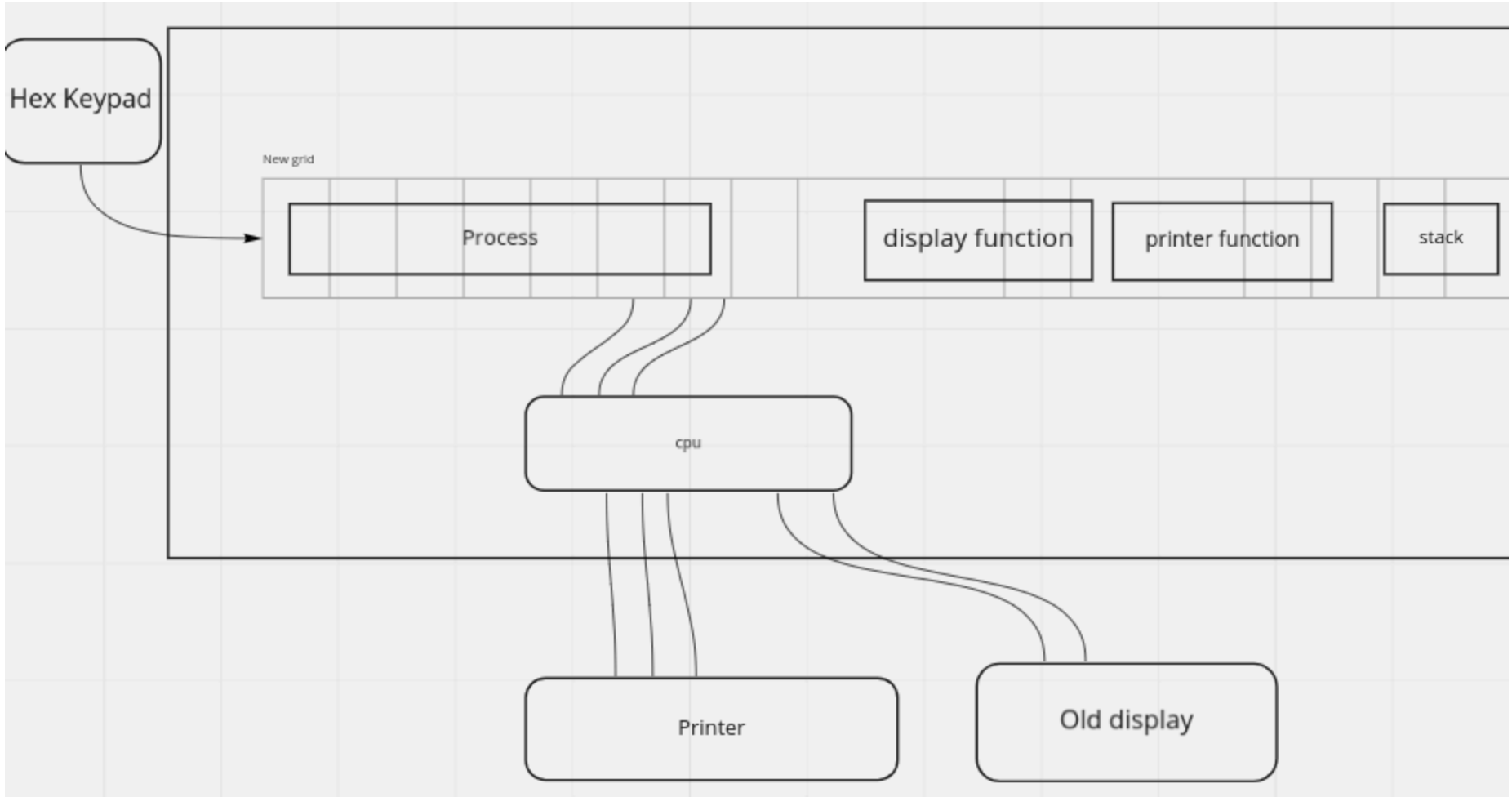
Jump to Printer Procedure



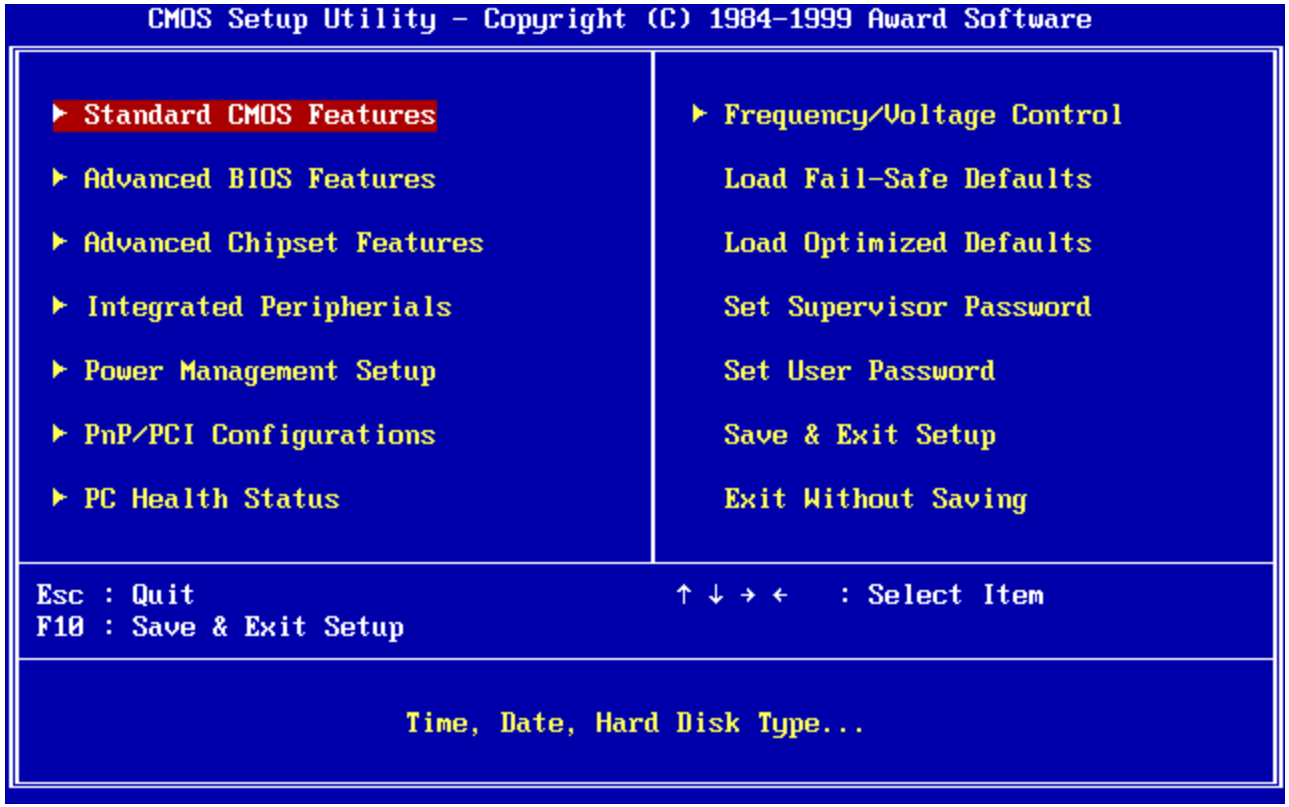
Return from Printer Procedure

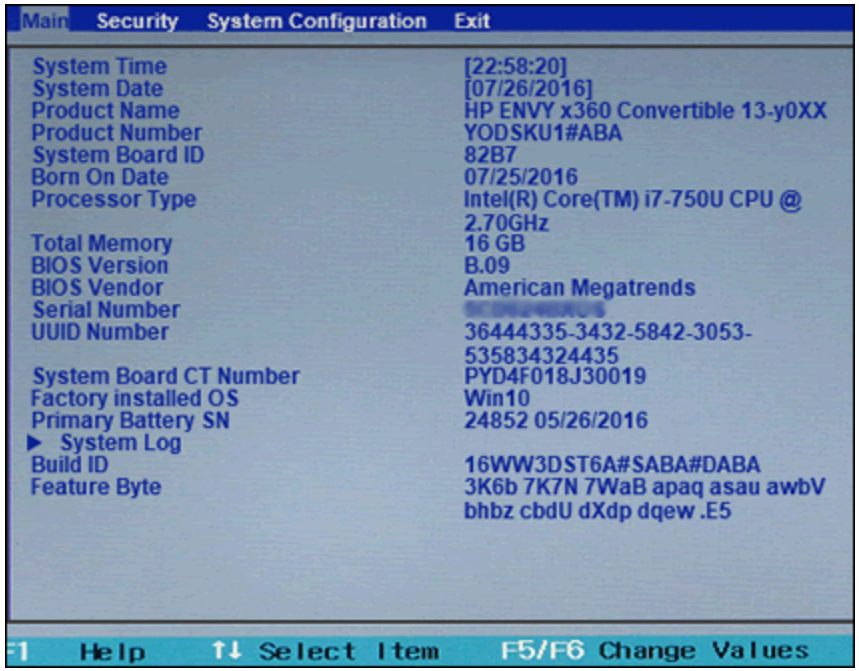


Display and Printer Procedure

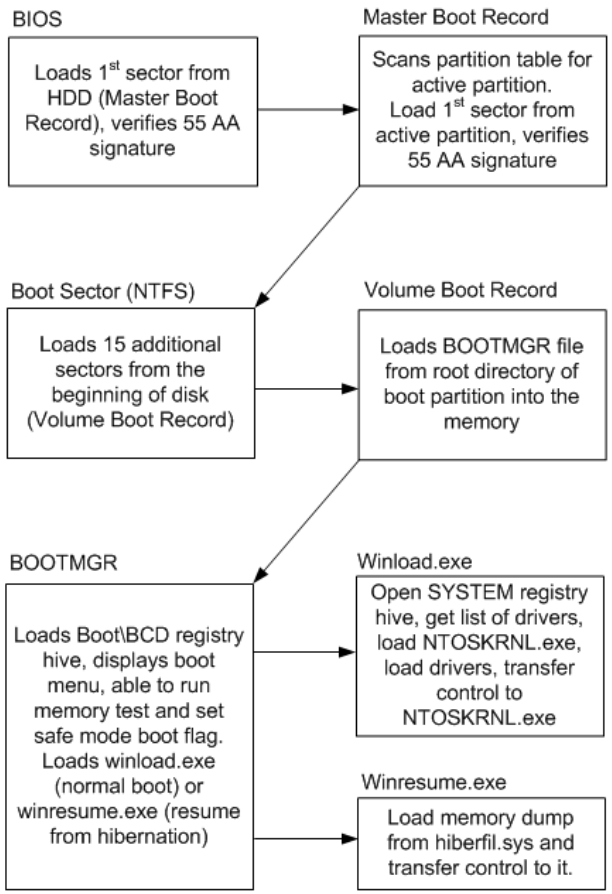


BIOS

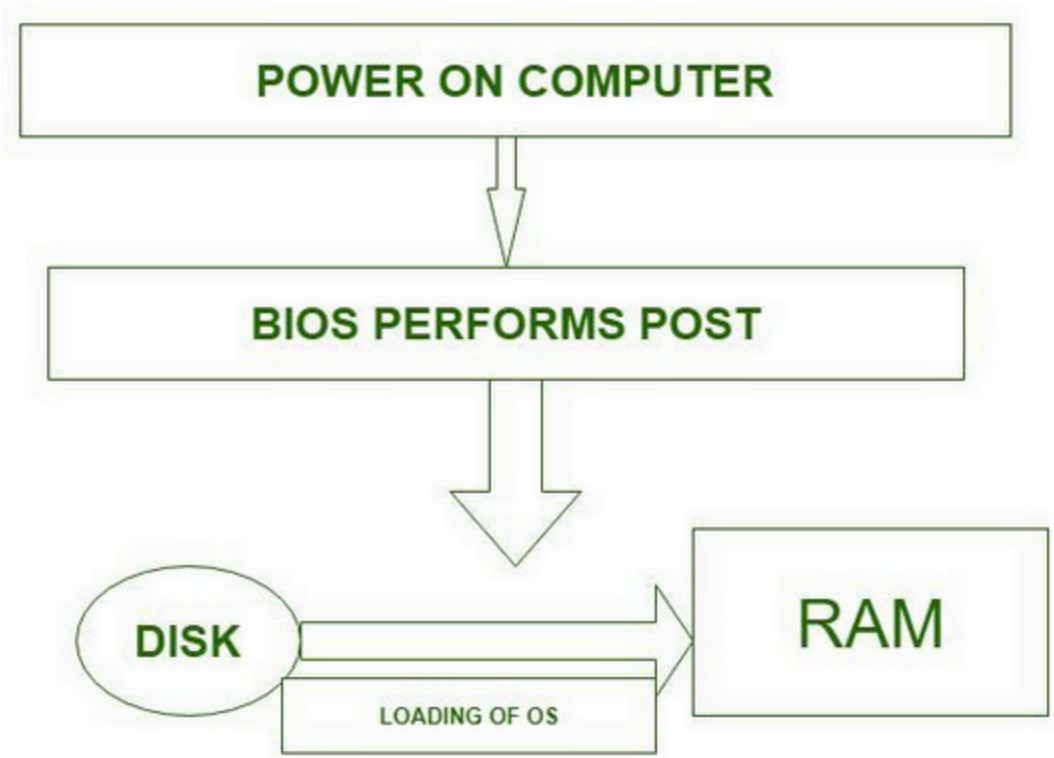

CMOS Setup Utility - Copyright (C) 1984-1999 Award Software
Standard CMOS Features
Advanced BIOS Features
Advanced Chipset Features
Integrated Peripherials
Power Management Setup
PnP/PCI Configurations
PC Health Status
Frequency/Voltage Control
Load Fail-Safe Defaults
Load Optimized Defaults
Set Supervisor Password
Set User Password
Save & Exit Setup
Exit Without Saving
Esc : Quit
F10 : Save & Exit Setup
↑ ↓ → ← : Select Item
Time, Date, Hard Disk Type...


Main Security System Configuration Exit
System Time [22:58:20]
System Date [07/26/2016]
Product Name HP ENVY x360 Convertible 13-y0XX
Product Number YODSKU1#ABA
System Board ID 82B7
Born On Date 07/25/2016
Processor Type Intel(R) Core(TM) i7-750U CPU @ 2.70GHz
Total Memory 16 GB
BIOS Version B.09
BIOS Vendor American Megatrends
Serial Number
UUID Number 36444335-3432-5842-3053-535834324435
System Board CT Number PYD4F018J30019
Factory installed OS Win10
Primary Battery SN 24852 05/26/2016
System Log
Build ID 16WW3DST6A#SABA#DABA
Feature Byte 3K6b 7K7N 7WaB apaq asau awbV bhbz cbdU dXdp dqew .E5
F1 Help ↑↓ Select Item F5/F6 Change Values

Boot sequence

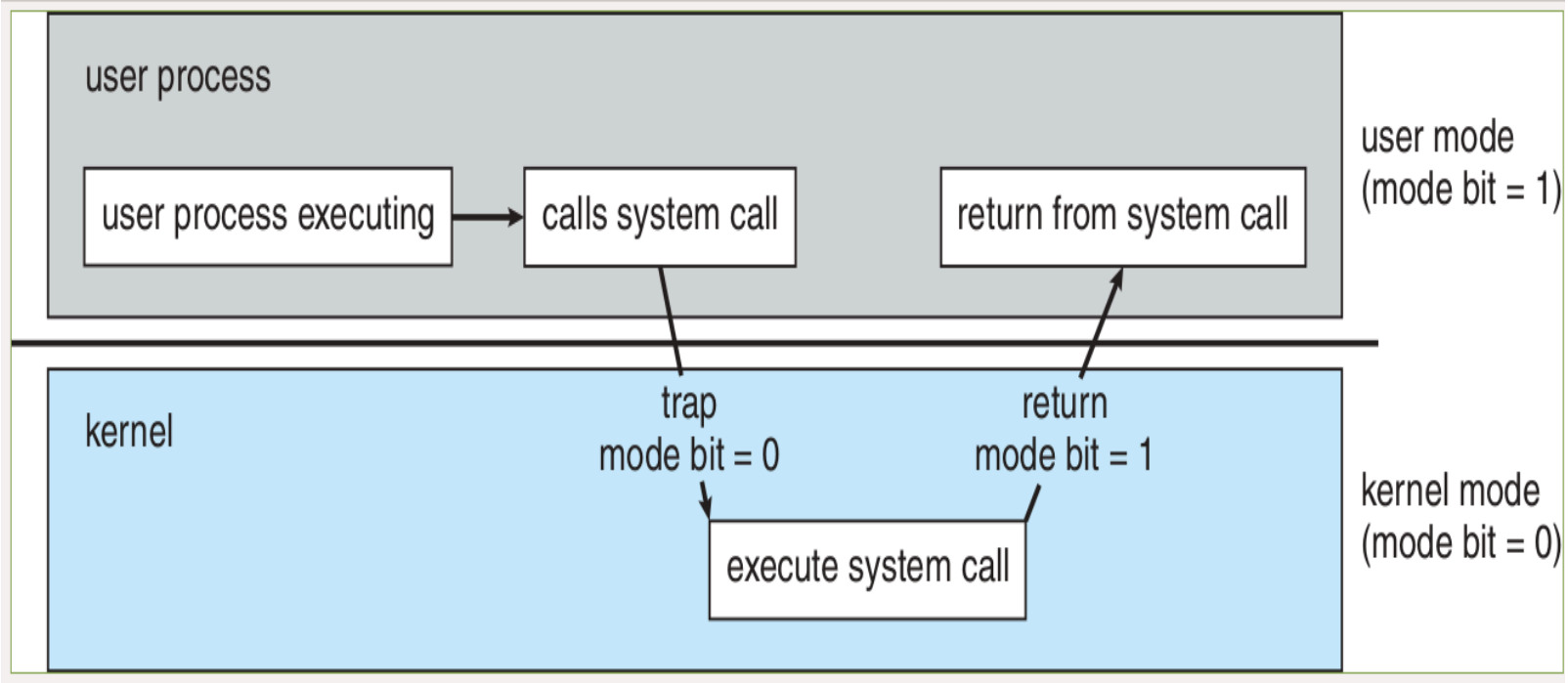


```
Ubuntu 8.04, kernel 2.6.24-16-generic
Ubuntu 8.04, kernel 2.6.24-16-generic (recovery mode)
Ubuntu 8.04, memtest86+
Other operating systems:
Windows Vista/Longhorn (loader)

Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the
commands before booting, or 'c' for a command-line.

The highlighted entry will be booted automatically in 4 seconds.
```

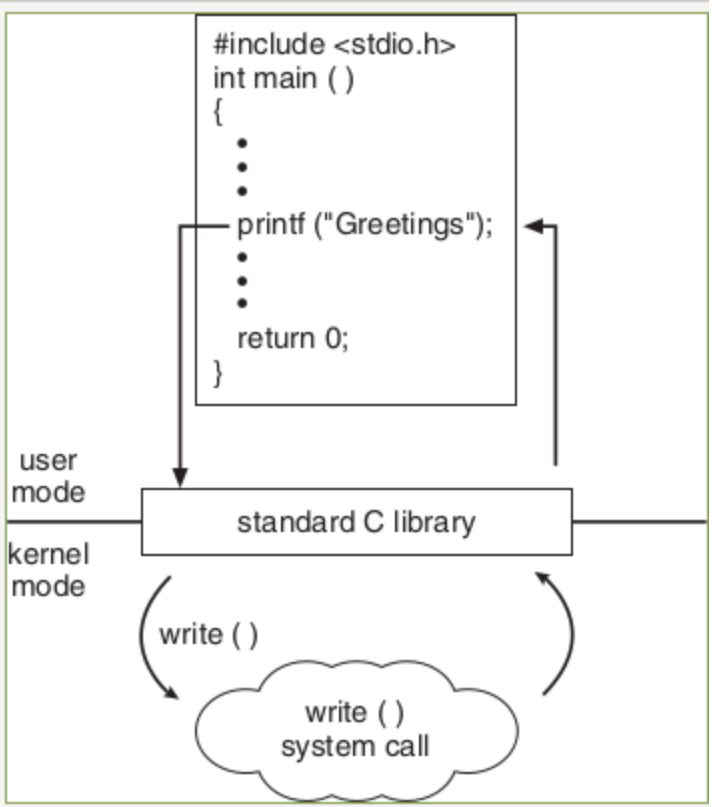

POWER ON COMPUTER
BIOS PERFORMS POST
DISK
RAM
LOADING OF OS

- IEEE Std 1275 1994 Standard for boot initialization
- https://openfirmware.info/Welcome_to_OpenBIOS
- https://github.com/openbios
- https://github.com/openbios/openbios

System Call

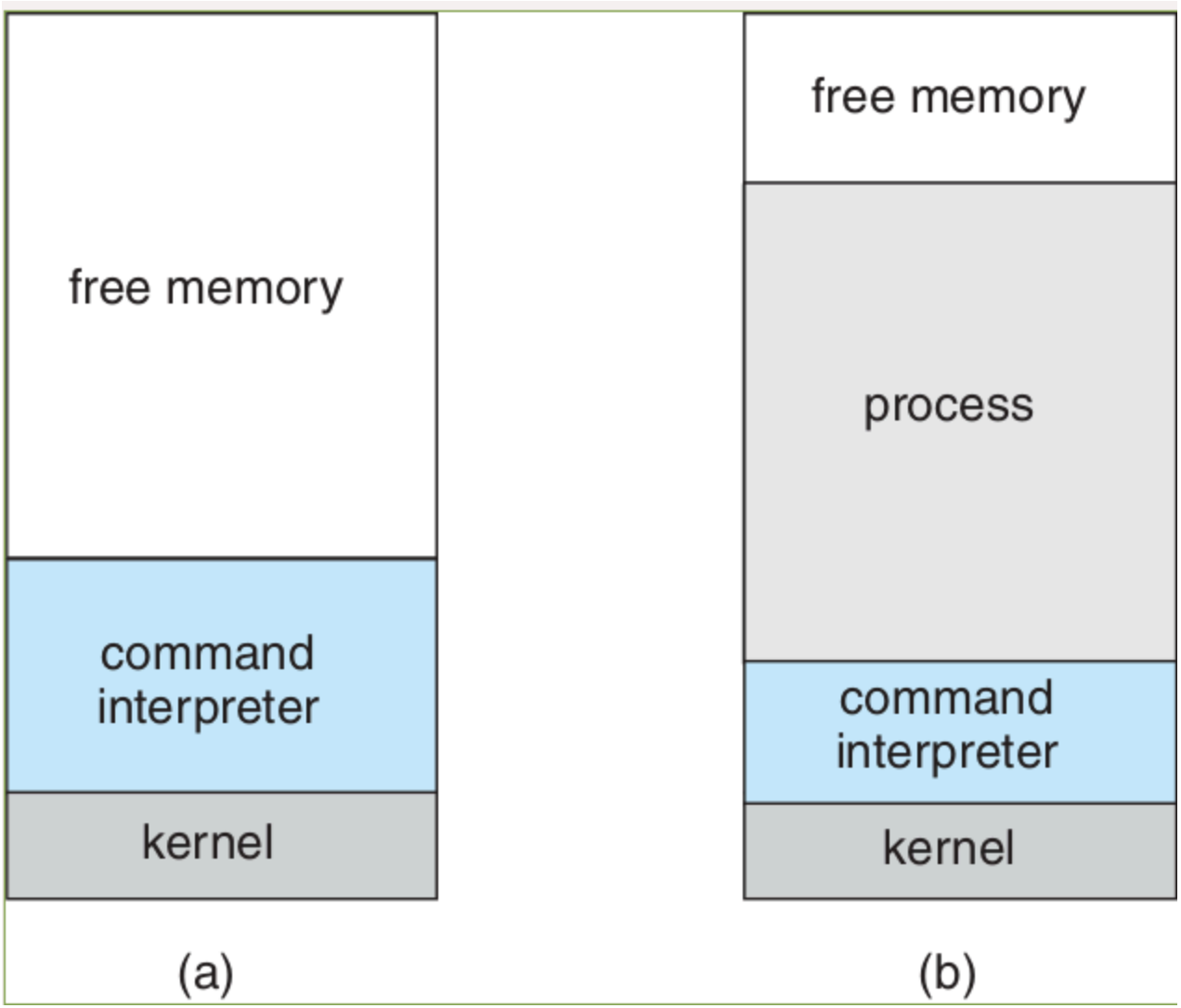

user process
user process executing
calls system call
return from system call
user mode
(mode bit = 1)
kernel
trap
mode bit = 0
return
mode bit = 1
execute system call
kernel mode
(mode bit = 0)

C System Call

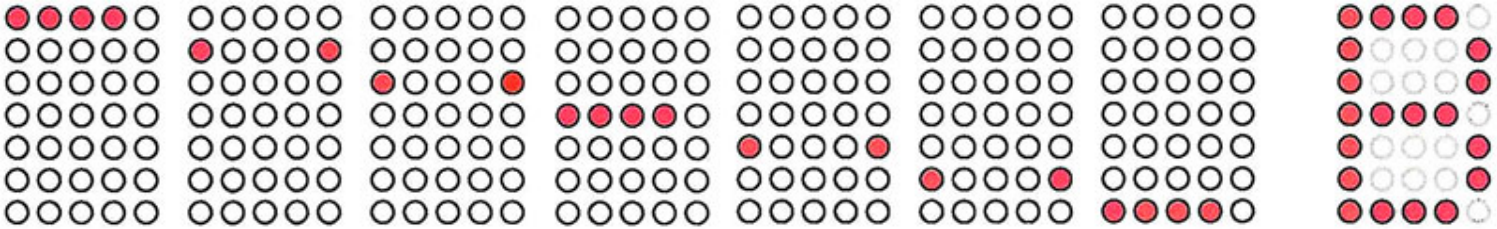


YIC120 - Adding Keyboard & Disk

- terminal (command prompt)
- batch system
- interactive system


free memory
command interpreter
kernel
(a)
free memory
process
command interpreter
kernel
(b)

When a controller rapidly turns on LEDs in one row at a time



https://www.nutsvolts.com/magazine/article/create-an-led-sign-controller

Context Switch

END